



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

**A Progress Report
On
NutriFit: An AI Based Dietary Recommendation System for General Wellness and
Chronic Diseases**

Submitted By:

Sakshyam Luitel	(078BCT073)
Sharad Pokharel	(078BCT083)
Sujan Thapaliya	(078BCT094)

Submitted To:

Department of Electronics and Computer Engineering
Pulchowk Campus
Lalitpur , Nepal

March, 2026

DECLARATION

We hereby declare that the report of the project entitled "NutriFit: An AI Based Dietary Recommendation System for General Wellness and Chronic Diseases" which is being submitted to the Department of Electronics and Computer Engineering, IOE, Pulchowk Campus, in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in Computer Engineering, is a bonafide report of the work carried out by us. The materials contained in this report have not been submitted to any University or Institution for the award of any degree and we are the only author of this complete work and no sources other than the listed here have been used in this work

Sakshyam Luitel (078BCT073) _____

Sharad Pokharel (078BCT083) _____

Sujan Thapaliya (078BCT094) _____

DATE: March,2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a project report entitled '**NutriFit: An AI Based Dietary Recommendation System for General Wellness and Chronic Diseases**' submitted by **Sakshyam Luitel, Sharad Pokharel, Sujan Thapaliya** in partial fulfillment of the requirements for the Bachelor's Degree in Computer Engineering.

Supervisor:

Department of Electronics and Computer Engineering
Institute of Engineering, Pulchowk Campus

External Examiner:

Project Co-ordinator: **Er.**

Department of Electronics and Computer Engineering
Institute of Engineering, Pulchowk Campus

Head of Department: **Er.**

Department of Electronics and Computer Engineering
Institute of Engineering, Pulchowk Campus

DATE OF APPROVAL: March, 2026

COPYRIGHT

The authors have agreed that the Library, Department of Electronics and Computer Engineering, Institute of Engineering, Pulchowk Campus may make this report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisors who supervised the project work recorded herein or in their absence, by the Head of the Department wherein the project report was done. It is understood that the recognition will be given to the authors of this project and to the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering in any use of the material of this report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, Institute of Engineering, Pulchowk Campus and authors' written permission is strictly prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Pulchowk Campus.

ACKNOWLEDGEMENT

This project is prepared in partial fulfilment of the requirement for for the the bachelor's degree in Computer Engineering. We owe our deepest gratitude to the Department of Electronics and Computer Engineering IOE, Pulchowk Campus for providing us with an opportunity to work on a major project as a part of our syllabus. We would also like to owe our gratitude to our supervisor, Bibha Sthapit, for her guidance. The experience of working on this project will surely enrich our technical knowledge and also gives experience of working on a project and also develop our team works skills to a great extent.

Sakshyam Luitel (078BCT073)

Sharad Pokheral (078BCT083)

Sujan Thapaliya (078BCT094)

ABSTRACT

The NutriFit project presents a robust, personalized dietary recommendation system designed to optimize individual health outcomes through applied machine learning. Leveraging an advanced XGBoost regression model, the system dynamically evaluates and predicts the nutritional suitability of various meals based on user-specific biometric profiles, underlying disease conditions, and dietary preferences. The backend architecture, built on Django, integrates a highly optimized inference engine that maps complex health constraints to a curated recipe dataset, while a responsive React frontend ensures an intuitive user experience. Quantitative evaluations using comprehensive regression metrics demonstrate the model's high precision in filtering and suggesting optimal meals. Ultimately, NutriFit bridges the gap between data-driven predictive modeling and practical dietary management, offering a scalable solution for personalized healthcare and nutritional tracking.

Keywords: Personalized Nutrition, Machine Learning, XGBoost, Dietary Recommendation System, Health Informatics, Predictive Modeling

TABLE OF CONTENTS

DECLARATION	i
CERTIFICATE OF APPROVAL	ii
COPYRIGHT	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	ix
LIST OF TABLES	x
1 LIST OF ABBREVIATIONS	xi
LIST OF ABBREVIATIONS	xi
2 INTRODUCTION	1
2.1 Background	1
2.2 Motivation	1
2.3 Objectives	2
2.4 Problem statement	2
2.5 Scope of Project	2
3 LITERATURE REVIEW	4
3.1 DASH Diet Recommendation for Hypertensive Patients	4
3.2 Overview of Recommender Systems in the Healthy Food Domain	4
3.3 Yum-Me: Personalized Nutrient-Based Meal Recommender	5
3.4 Disease-Aware Dietary Recommendation	5
3.5 IoMT-Based Real-Time Dietary Monitoring and Recommendation	6
4 REQUIREMENT ANALYSIS	8
4.1 Functional Requirements	8
4.2 Non-Functional Requirements	9
4.3 Hardware Requirements	9
4.4 Software Requirements	10
5 SYSTEM ARCHITECTURE AND METHODOLOGY	12
5.1 Theoretical Background	12

5.2	Architecture and Algorithms Used for the Project	12
5.3	System Architecture Diagram	13
5.4	Use Case Diagram	14
5.5	Activity Diagram	14
5.6	Class Diagram for System	16
5.7	Class Diagram for Data	16
5.8	Sequence Diagram	17
5.9	Communication Diagram	18
5.10	Data Flow Diagrams	19
5.10.1	Level 0: Context Diagram	19
5.10.2	Level 1: System Sub-Processes	19
5.10.3	Level 2: Inference Engine Controller Workflow	20
5.10.4	Level 2: XGBoost Regressor Internal Pipeline	20
5.11	Deployment Diagram	21
6	IMPLEMENTATION DETAILS	23
6.1	Frontend Application Implementation	23
6.2	Backend Controller Implementation	24
6.3	Machine Learning Pipeline: Data Collection and Preprocessing	24
6.3.1	Data Collection Methodology	24
6.3.2	Data Preprocessing and Synthetic Bootstrapping	25
6.4	Machine Learning Pipeline: Model Training and Evaluation	27
6.4.1	Evaluation Summary	32
7	RESULTS AND DISCUSSIONS	33
7.1	Graphical User Interface Outputs	33
7.2	Nutritional Output Analysis & Visualizations	34
7.3	Model Performance and Error Analysis	37
7.3.1	Metric Diagnostics	37
7.4	Sources of Error and Deviations	38
8	CONCLUSION	39
9	LIMITATIONS AND FUTURE ENHANCEMENTS	40
9.1	Limitations	40
9.2	Future Enhancements	41
	REFERENCES	42

List of Figures

5.1	Overall System Architecture of NutriFit	13
5.2	Use Case Diagram of NutriFit System	14
5.3	Activity Diagram tracking User and System Flow	15
5.4	System Class Diagram mapping Application Tiers	16
5.5	Data Class Diagram outlining Object Relationships	17
5.6	Sequence Diagram of the Recommendation Engine process	18
5.7	Communication Diagram between primary Objects	18
5.8	Level 0 DFD: System Context Diagram	19
5.9	Level 1 DFD: Primary System Modules and Control Flow	20
5.10	Level 2 DFD: Internal Inference Engine Aggregation and Ranking	20
5.11	Level 2 DFD: Mathematical Workflow of the XGBoost Pipeline	21
5.12	Deployment Diagram illustrating Server and Client Nodes	22
6.1	Histographical distribution of Recipe Calories vs Protein outputted during the Exploratory Data Analysis (EDA) preprocessing phase.	27
6.2	Three-panel XGBoost evaluation: Actual vs. Predicted scatter (left), residual distribution centered at zero (centre), and Top 15 feature importances by gain (right).	28
6.3	Top 20 XGBoost feature importances visualized with a viridis color gradient. <code>calorie_deficit</code> ranks highest (≈ 0.148), followed by <code>bmi_obese</code> (≈ 0.093) and <code>disease_fat_influence</code> (≈ 0.091).	29
6.4	XGBoost learning curves over 300 boosting rounds for RMSE (left), R^2 (centre), and MAE (right). Train and Test curves overlap and converge by round 50 to $RMSE \approx 0.050$, $R^2 \approx 0.91$, and $MAE \approx 0.041$	29
6.5	Ground Truth vs. Predicted scores on held-out test samples against the ideal diagonal. Points cluster tightly along the diagonal, with wider spread at lower scores (0.15–0.40) due to higher label variance in low-suitability edge cases. . .	30
6.6	5-Fold Cross-Validation R^2 (bars) versus Train $R^2 = 0.917$ (green line), Test $R^2 = 0.910$ (red dashed), and Mean CV $R^2 = 0.911$ (orange dash-dot). All folds fall between 0.909 and 0.911.	31
6.7	Rigorous Ranking Evaluation: Strict Precision@k at top-1% threshold (blue, left axis) and NDCG@k (orange dashed, right axis) across $k \in \{5, 10, 20, 50, 100\}$. Precision rises from 0.60 at $k = 5$ to ≈ 0.86 at $k \geq 20$; NDCG@k remains near 1.0 throughout.	31
7.1	Output of predicted dietary recommendations on the Recipe Page.	33
7.2	User Cart interface tracking selected recipes and cumulative nutritional values.	34

7.3	Comparison between theoretically required calories and the actual cumulative calories of the selected diet.	35
7.4	Macronutrient distribution (Carbohydrates, Protein, Fat) of the resulting diet plan.	36

List of Tables

6.1	NutriFit XGBoost Model — Consolidated Evaluation Metrics	32
7.1	Testing and Training Evaluation Metrics of the XGBoost Regressor	37

1. LIST OF ABBREVIATIONS

ACF Auto-correlation Function
AI Artificial Intelligence

2. INTRODUCTION

2.1. Background

The intersection of artificial intelligence and healthcare has catalyzed a paradigm shift in how individuals approach preventative medicine and daily well-being. Nutrition is universally recognized as a foundational pillar of preventative health, directly influencing the onset, progression, and management of chronic lifestyle-induced diseases such as type 2 diabetes, essential hypertension, and hyperlipidemia. Historically, dietary guidelines have been disseminated as generalized, population-level advice, relying heavily on broad food pyramids or static caloric targets. These traditional approaches fail to account for the highly individualized nature of human metabolism, patient-specific biometric profiles, and the nuanced nutritional constraints imposed by pre-existing pathological conditions. With the proliferation of comprehensive health datasets and the advancement of predictive machine learning algorithms, modern computational frameworks can now map multidimensional health parameters to optimal dietary choices. This represents a critical evolution from generalized dietetics to highly personalized, data-driven nutritional informatics.

2.2. Motivation

The conceptualization of NutriFit is driven by the stark divide between clinical dietary prescriptions and practical, daily culinary execution. When patients are diagnosed with diet-sensitive conditions, they are often burdened with complex matrices of macronutrient and micronutrient restrictions that are overwhelmingly difficult to translate into everyday meal planning. The cognitive load required to cross-reference daily recipe choices against complex medical guidelines frequently leads to dietary non-compliance, frustration, and deteriorating health outcomes. Furthermore, the majority of commercially available health applications focus heavily on retroactive tracking, acting merely as food diaries rather than proactive, prescriptive intelligence. The primary motivation is to build a bridge across this gap by engineering an intelligent, autonomous system that absorbs the friction of dietary planning. By leveraging machine learning to automatically evaluate and predict the suitability of thousands of recipes against strict, individualized biometric parameters, NutriFit aims to empower vulnerable populations to make safe, medically compliant culinary decisions with confidence and ease.

2.3. Objectives

The primary goals of this project are concentrated into two core technical and systemic achievements:

1. To engineer, train, and deploy a robust predictive machine learning pipeline by utilizing an advanced XGBoost regression architecture—capable of accurately evaluating and scoring the nutritional suitability of daily meals against highly specific, localized user biometric profiles and multi-disease constraints.
2. To seamlessly integrate this predictive inference engine within a scalable, modern full-stack web application (Django backend and React frontend), providing an automated, highly responsive platform that translates complex algorithmic outputs into intuitive, actionable dietary recommendations for end-users.

2.4. Problem statement

Despite the abundance of health tracking technology, standardized nutritional platforms remain fundamentally inadequate for individuals actively managing specific, diet-restrictive medical conditions. The prevailing methodology relies on user driven calorie counting and manual food logging, which is inherently flawed for preventative care. The core computational problem lies in the absence of an accessible, algorithmic bridge capable of calculating the multi-dimensional suitability of a given meal prior to consumption. Without a system to autonomously predict how an exact recipe interacts with intersecting medical constraints (e.g., managing high cholesterol simultaneously with diabetes), individuals face an unmanageable knowledge gap, leaving them vulnerable to dietary mistakes, malnutrition, or severe non-compliance with critical clinical restrictions.

2.5. Scope of Project

The scope of the NutriFit project encompasses the end-to-end development of a specialized machine learning model and its operational deployment through a full-stack web ecosystem. The technical boundary includes the pre-processing of curated dietary datasets, the training and tuning of an XGBoost regression model for nutritional prediction, and the deployment of a Django based REST API to serve predictions. The user facing scope is restricted to generating proactive, mathematically scored recipe recommendations based on static biometric inputs (Age, Weight, Height, BMI, Activity Level) and specific categorical diseases.

Conversely, the boundary of this project deliberately excludes real time biomedical integration (such as syncing with IoT continuous glucose monitors or smartwatches), HIPAA-compliant electronic medical record (EMR) integrations, and broader physical fitness or pharmaceutical counseling. NutriFit operates strictly as a supplemental dietary informatics tool and does not substitute formalized diagnosis or clinical treatment by a certified medical practitioner.

3. LITERATURE REVIEW

This chapter contains all the existing works that have already been carried out in the field related to this project topic. It details the extensive research undertaken to understand the current state-of-the-art in dietary recommendation systems, identifying key methodologies, limitations, and the foundational context that validates the development of the NutriFit system.

3.1. DASH Diet Recommendation for Hypertensive Patients

Sookrah, Dhowtal, and Nagowah [1] developed a machine learning-based dietary recommendation system specifically designed for patients suffering from hypertension, utilizing the DASH (Dietary Approaches to Stop Hypertension) diet as its core clinical framework. The system operates by accepting basic patient demographic data and deploying a Naïve Bayes classifier to categorize the suitability of various meals against established DASH metrics. This work is highly important as it empirically demonstrates that supervised machine learning models can be successfully applied to clinical dietary guidance, proving that disease-specific databases vastly improve recommendation safety over generic datasets. However, a major drawback is its narrowly restricted scope; the system exclusively targets a single condition (hypertension) and fails to account for fundamental metabolic calculations like Basal Metabolic Rate (BMR) or Total Daily Energy Expenditure (TDEE). By restricting its functionality to a single disease and omitting core metabolic markers, the system is too brittle to function as a holistic healthcare tool, providing inadequate recommendations for users with concurrent conditions. This limitation directly motivates the development of NutriFit, which scales disease coverage to 120 distinct conditions and computes individualized metabolic requirements, allowing the model to serve highly complex, multi-symptomatic user profiles effectively.

3.2. Overview of Recommender Systems in the Healthy Food Domain

Tran, Atas, Felfernig, and Stettinger [2] conducted a comprehensive theoretical survey of existing recommender system approaches applied to healthy food choices. Categorizing food recommendation algorithms into content-based, collaborative, and hybrid approaches, the research evaluates their architectural strengths and theoretically proposes "weighted constraint encoding" as an innovative method for incorporating strict dietary restrictions. This paper provides critical theoretical grounding for combining machine learning personalization

with rigid dietary constraints, explicitly identifying industry hurdles such as the cold-start problem. Nevertheless, as a survey paper, its most glaring limitation is the lack of a functional implementation; it strictly evaluates existing platforms without empirically validating its proposed theoretical solutions in a production environment. A theoretical framework alone cannot solve the practical friction users face, leaving the real-world computational efficiency of weighted constraints unproven. This gap between theoretical research and practical execution heavily informed the architectural design of NutriFit, which takes the theoretical concept of weighted constraints and physically implements them as continuous influence values within a live XGBoost architecture, proving that complex clinical restrictions can be computationally served in a real-world web application.

3.3. Yum-Me: Personalized Nutrient-Based Meal Recommender

Yang et al. [3] introduced Yum-Me, an advanced personalized meal recommender system focused on customized nutrient-based diet planning. Yum-Me incorporates rigorous metabolic calculations derived from established dietary reference intakes, allowing users to define explicit nutrient preferences and utilizing deterministic, rule-based matching algorithms to generate daily meal plans. This platform successfully demonstrates the physiological value of combining user-specific metabolic data with raw food nutrient profiles to force measurable improvements in dietary goal adherence. However, its central limitation is the complete lack of predictive machine learning; recommendations represent basic algebraic matching rather than intelligent scoring. The system is purely built for macronutrient balancing and lacks the logic to account for pathological disease conditions, placing the burden of knowing which "nutrient targets" are medically safe squarely on the user. By relying on strict deterministic arithmetic, Yum-Me functions more as an advanced dietary calculator than an intelligently adaptable recommender. NutriFit directly builds upon Yum-Me's successful metabolic computation baseline but completely replaces its rigid rule-based matching with an XGBoost machine learning model, allowing it to autonomously predict and score meals based on underlying medical constraints without forcing users to manually calculate their safe nutrient targets.

3.4. Disease-Aware Dietary Recommendation

Toledo et al. [4] researched computational methodologies for incorporating disease-specific dietary constraints into food recommendation systems, heavily focusing on cardiovascular conditions and type 2 diabetes. The system physically maps established clinical guidelines to strict nutrient restriction rules, which are then applied as rigid, hard-coded filters across

a broad recipe database to eliminate non-compliant meals. This work plays a vital role in empirically validating that disease-aware filtering mechanisms reliably produce dietary recommendations that are significantly safer than general healthy eating advice for vulnerable populations. However, the reliance entirely on static, hard-rule-based filtering devoid of any machine learning elasticity presents a critical drawback. Hard-coded rules are incredibly brittle and fail to understand the continuous, multi-dimensional nature of nutrition; a meal exceeding a clinical bound by a single milligram is rejected outright, even if it optimally balances other crucial macronutrients. This inability to learn complex, non-linear trade-offs between differing health metrics motivated NutriFit’s adoption of advanced gradient boosting algorithms. By utilizing an XGBoost model, NutriFit captures the highly complex, non-linear feature interactions that static rule-based systems simply cannot express, ensuring users receive elastic, intelligently ranked suitability scores rather than encountering an absolute pass/fail barrier.

3.5. IoMT-Based Real-Time Dietary Monitoring and Recommendation

Sharma et al. [5], in a study published by IEEE, proposed a comprehensive Internet of Medical Things (IoMT) framework designed for real-time dietary monitoring and predictive recommendation. The system works by actively integrating continuous data streams from complex wearable biomedical sensors—such as continuous glucose monitors and autonomic nervous system trackers—with a cloud-based Artificial Neural Network (ANN). This intensive architecture dynamically calculates and recommends meals based on the user’s real-time physiological fluctuations. The importance of this research is substantial, as it demonstrates the bleeding edge of preventative healthcare interventions, allowing for immediate, microscopic dietary course corrections for high-risk patients managing volatile conditions. However, the system’s absolute reliance on continuous sensor integration presents severe drawbacks; it requires expensive proprietary hardware, demands constant low-latency internet connectivity, and suffers from heavy battery and hardware dependencies. A major criticism of this approach is the massive socio-economic barrier to entry it creates, alongside significant electronic health record (EHR) privacy and security vulnerabilities inherent in constantly transmitting live biomedical telemetry. Furthermore, forcing patients to wear intrusive physical sensors 24/7 often leads to severe user fatigue and subsequent system abandonment. The brittleness and hardware-exclusivity of this IEEE-published framework directly shape the primary motivation behind NutriFit. NutriFit deliberately decouples intelligent dietary prediction from live hardware dependencies, opting instead for a highly accessible, entirely software-based application. By utilizing robust XGBoost algorithms applied to static user biometric inputs and known disease classifications [6, 7, 8, ?] rather than requiring expen-

sive active sensor data, NutriFit ensures that advanced, personalized nutritional guidance remains universally accessible and affordable, democratizing access to clinical-grade dietary intelligence without gating it behind expensive IoMT physical devices.

4. REQUIREMENT ANALYSIS

This section outlines the specific functional, non-functional, hardware, and software requirements necessary for the development, training, deployment, and end-user execution of the NutriFit system. These requirements ensure that the system operates efficiently, from handling complex machine learning computations during model training to providing a seamless, responsive user interface.

4.1. Functional Requirements

Functional requirements define the core behaviors, features, and capabilities the system must execute to successfully fulfill the user's objectives.

- **User Profiling & Biometric Evaluation:** The system must allow users to input physical biometric details (height, weight, age, sex, activity level) and algorithmically calculate their resulting Basal Metabolic Rate (BMR) and Total Daily Energy Expenditure (TDEE).
- **Dietary Goal Configuration:** The system must permit users to define broad nutritional objectives (weight loss, weight maintenance, or muscle gain) to constrain target caloric intake dynamically.
- **Disease Management Integration:** The system must provide an interface for users to select pre-existing medical conditions (e.g., diabetes, hypertension) and mathematically enforce absolute dietary restrictions (e.g., restricting sugar or sodium thresholds).
- **Algorithmic Recipe Recommendation:** The integrated Machine Learning engine must dynamically ingest user constraints, evaluate the dataset, and return the optimal top-ranked recipes tailored independently to specific meal types (breakfast, lunch, dinner).
- **Data Search and Retrieval:** The platform must structurally search, filter, and extract nutritional metadata and recipe instructions from an extensive backend dataset seamlessly.

4.2. Non-Functional Requirements

Non-functional requirements define the critical quality attributes, constraints, and performance standards the system must uphold during execution.

- **Performance and Latency:** The recommendation engine must run vectorized batch predictions and return complex recipe lists efficiently, aiming for rapid execution times (e.g., fractional seconds per batch query) to prevent UI blockages.
- **High Scalability:** The backend infrastructure (SQLite database logic) and the serialized XGBoost model must be structured to successfully house and evaluate a massive recipe matrix (exceeding hundreds of thousands of rows) without catastrophic memory failures or freezing.
- **Algorithmic Reliability:** The machine learning pipeline must maintain heavily validated mathematical accuracy (high Precision@k and low RMSE) to guarantee that provided meal recommendations remain strictly safe for individuals managing severe, chronic health conditions.
- **Usability and Responsiveness:** The frontend Web Client must present a highly intuitive, accessible user interface capable of collapsing complex dietary matrices into an easily navigable design that remains functionally responsive across all major screen sizes.

4.3. Hardware Requirements

The hardware requirements are divided into two distinct categories: the development environment used to build and train the machine learning models, and the client environment required by the end-user to access the application.

1. Development Environment

The development and training of the XGBoost regression model, along with the simultaneous processing of the backend and frontend servers, require a moderately capable personal computing system.

- **Processor (CPU):** Minimum multi-core processor (e.g., Intel Core i5 or AMD Ryzen 5). A capable CPU is strictly required to parallelize the gradient boosting processes during the XGBoost model training phase and to handle rapid local server recompilations.

- **Memory (RAM):** Minimum 8GB (16GB recommended). Substantial memory capacity is required in the backend to load and manipulate the extensive curated recipe datasets (CSV/JSON formats) into Pandas DataFrames without causing memory overflow errors when fitting the machine learning model.
- **Storage:** At least 10GB of free Solid State Drive (SSD) storage. High-speed storage is required to ensure fast read/write execution when the system saves and loads the massively serialized XGBoost model files (.joblib and .json), and when reading local nutritional datasets into memory.

2. End-User Environment

Because NutriFit is a web-based application utilizing a client-server architecture, the hardware requirements for the end-user are intentionally minimal.

- **Device:** Any internet-capable device, such as a smartphone, tablet, laptop, or desktop computer. The device is required simply to execute basic JavaScript, render the HTML/CSS user interface, and transmit HTTP requests. All heavy computational logic and machine learning inference computations are completely offloaded to the backend server.

4.4. Software Requirements

The software stack for NutriFit is deeply integrated across the frontend web client, the backend server architecture, and the sophisticated machine learning pipeline.

1. Backend and Database Requirements

- **Python (3.8+):** The core programming language required to orchestrate the internal logic of the backend server and compile the multi-layered machine learning inference scripts.
- **Django (4.2+):** A high-level Python web framework required to structurally manage REST API routing, user authentication, session management, and secure HTTP communication between the React client and the predictive inference engine.
- **SQLite3:** The default relational database management system required to persistently and securely store user profiles, biometric inputs, defined dietary constraints, and the rigorously structured recipe dataset without the massive overhead of a dedicated external database server.

2. Machine Learning Pipeline Requirements

- **Scikit-Learn & XGBoost:** The primary machine learning libraries strictly required within the Python environment to architect, train, evaluate, and serialize the gradient boosting regression algorithms responsible for scoring complex recipe viability.
- **Pandas & NumPy:** Fundamental mathematical and data analysis libraries required within the backend data-processing logic to scrub, normalize, and manipulate raw tabular nutritional data into structured multidimensional arrays for ingestion by the predictive algorithm.
- **Jupyter Notebook:** An isolated, interactive computational environment required exclusively during the initial research and development phase to safely experiment with feature engineering, test the model's custom R^2 precision metrics, and visually plot training loss outcomes.

3. Frontend Development Requirements

- **Node.js & npm (Node Package Manager):** The runtime environment and package manager required to install essential frontend dependencies, compile the Vite build system, and run the local React development server.
- **React (18+):** A highly modular JavaScript library required to build the dynamic, Single-Page Application (SPA) architecture of the user interface. It facilitates instantaneous state updates (such as generating dynamic recipe feed suggestions) without requiring traditional browser page reloads.
- **Tailwind CSS:** A utility-first CSS framework required to rapidly design, layout, and style complex UI components, ensuring the platform remains uniformly responsive across both desktop monitors and mobile displays.

5. SYSTEM ARCHITECTURE AND METHODOLOGY

This section outlines the theoretical foundation of the NutriFit system and details the structural architecture, machine learning algorithms, and system methodologies used to build the platform.

5.1. Theoretical Background

The theoretical foundation of NutriFit rests on the intersection of data-driven computational nutrition and predictive machine learning. Unlike traditional dietetics, which relies on generalized heuristics, this system adopts a precision-medicine framework. It treats nutritional recommendation as a multi-dimensional regression problem, where independent variables (user biometrics and categorical diseases) are mathematically mapped to a dependent variable (meal algorithmic suitability score). The system also relies on the paradigm of decoupled application architecture, separating the client presentation layer (React) from the heavy computational server logic (Django), allowing the machine learning inference engine to operate entirely asynchronously from the user interface.

5.2. Architecture and Algorithms Used for the Project

NutriFit utilizes a three-tier architecture: a React-based frontend, a Django-based REST API backend, and an embedded Machine Learning inference engine.

The core algorithm driving the system is the **XGBoost (eXtreme Gradient Boosting) Regressor**. XGBoost is an ensemble learning method that builds continuous decision trees sequentially, where each new tree corrects the residual errors of the previous ones. The algorithm evaluates a complex matrix of features:

- **User Inputs:** BMI (calculated from height/weight), Age, and active Disease constraints.
- **Recipe Profiles:** Calories, Protein, Fat, Carbohydrates, and Sodium.

The model applies dynamic constraint weighting, effectively lowering the predicted suitability score of recipes that violate a user's medical thresholds (e.g., highly penalizing high-sugar recipes for diabetic profiles).

5.3. System Architecture Diagram

The overall architecture of the NutriFit system is based on a three-tier structure consisting of the presentation layer, application layer, and data layer.

The React frontend interacts with the Django backend through REST APIs. The backend processes user inputs, communicates with the database, and invokes the machine learning inference engine to generate dietary recommendations. The results are then returned to the frontend for visualization.

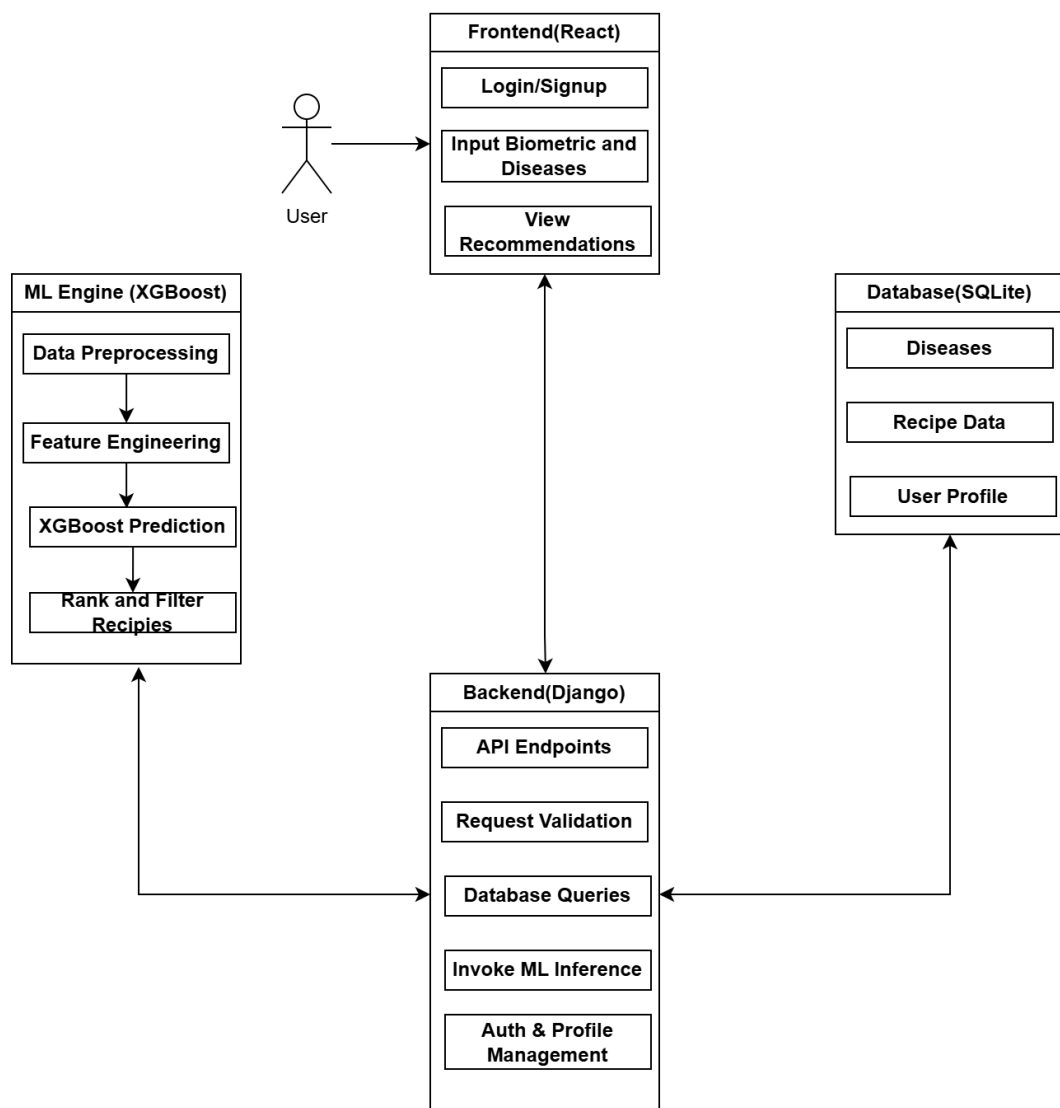


Figure 5.1: Overall System Architecture of NutriFit

5.4. Use Case Diagram

The Use Case diagram illustrates the primary interactions between the end-user and the NutriFit system, encompassing account management, biometric input, and the core recommendation generation process.

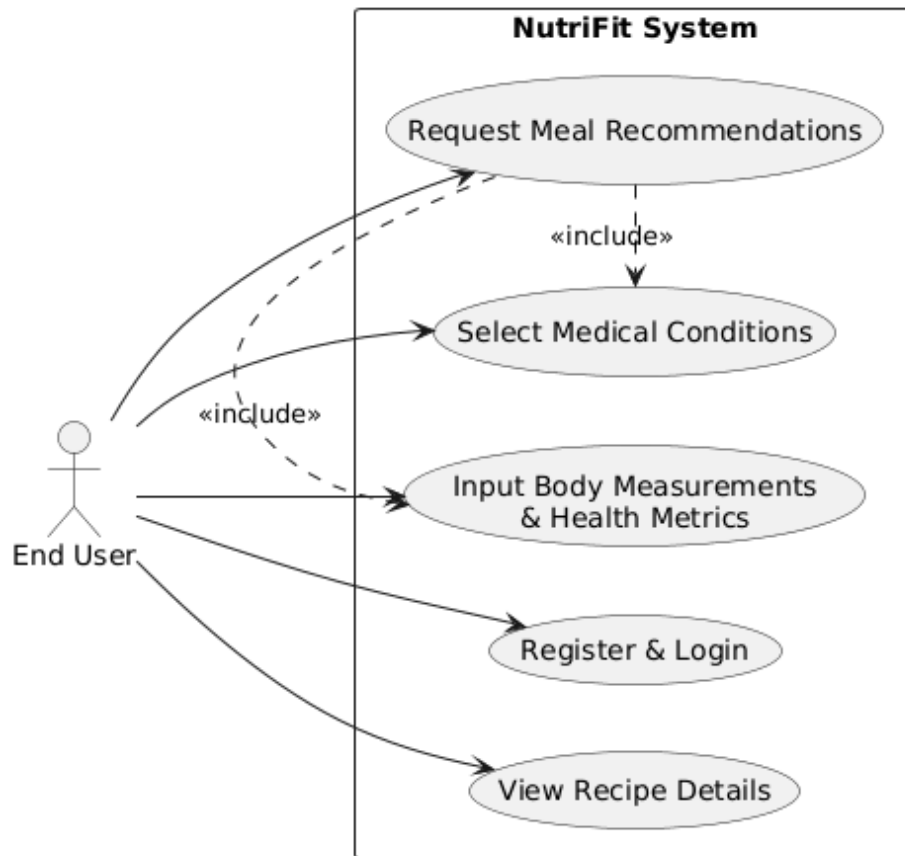


Figure 5.2: Use Case Diagram of NutriFit System

5.5. Activity Diagram

The Activity diagram models the procedural flow of the system from the moment the user logs in to the generation of their personalized dietary feed, including the condition checks for missing biometric parameters.

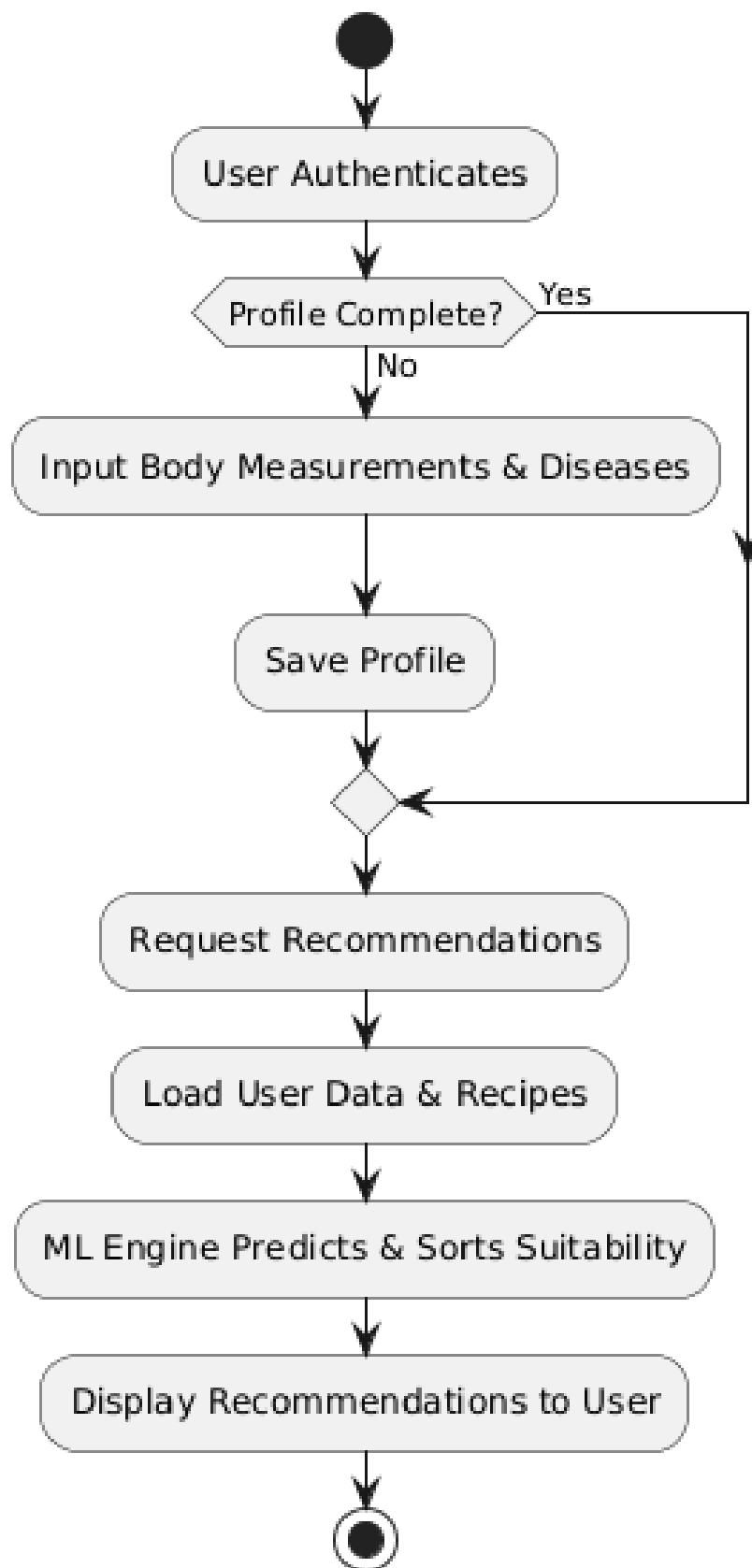


Figure 5.3: Activity Diagram tracking User and System Flow

5.6. Class Diagram for System

This diagram represents the structural software components of the system, segregated into the Frontend UI layer, API Backend Controllers layer, and the nested Machine Learning Inference Engine layer.

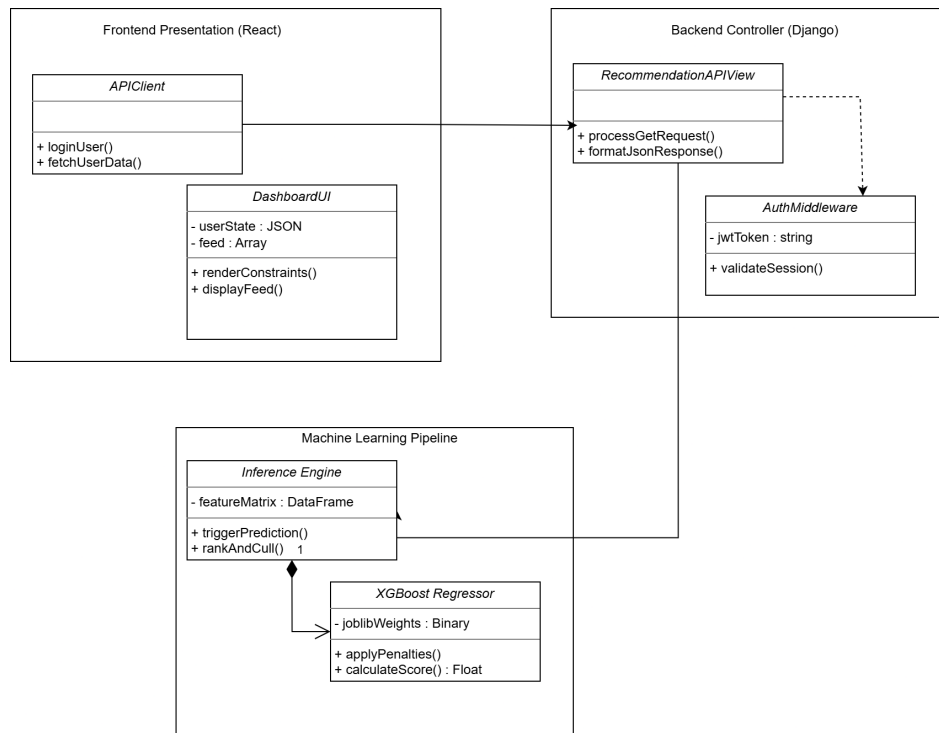


Figure 5.4: System Class Diagram mapping Application Tiers

5.7. Class Diagram for Data

The Data Class diagram highlights the distinct informational entities utilized by the application, focusing on the cardinality and relationships between the User profiles, defined Diseases, and nutritional Recipes.

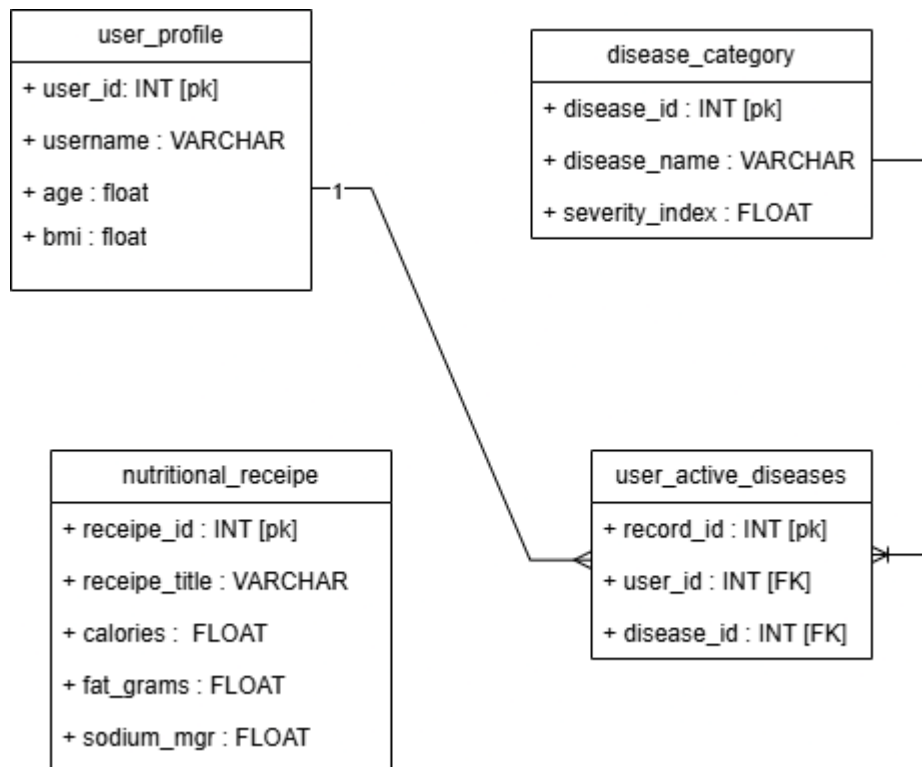


Figure 5.5: Data Class Diagram outlining Object Relationships

5.8. Sequence Diagram

The Sequence diagram models the chronological execution of a recipe recommendation request, explicitly mapping the lifecycles and return states between the User, React UI, Django API, Database, and ML model.

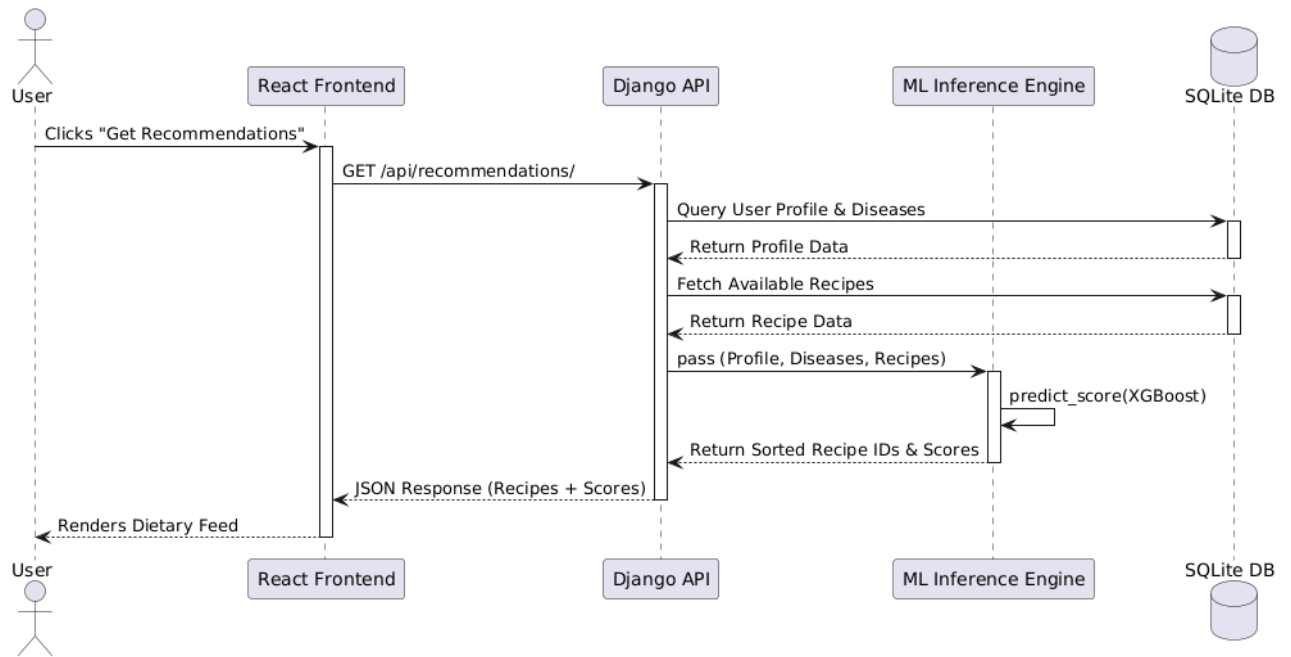


Figure 5.6: Sequence Diagram of the Recommendation Engine process

5.9. Communication Diagram

The Communication Diagram highlights the interconnected message paths and explicit structural relationships required to resolve an inference query, emphasizing component networking over strict chronology.

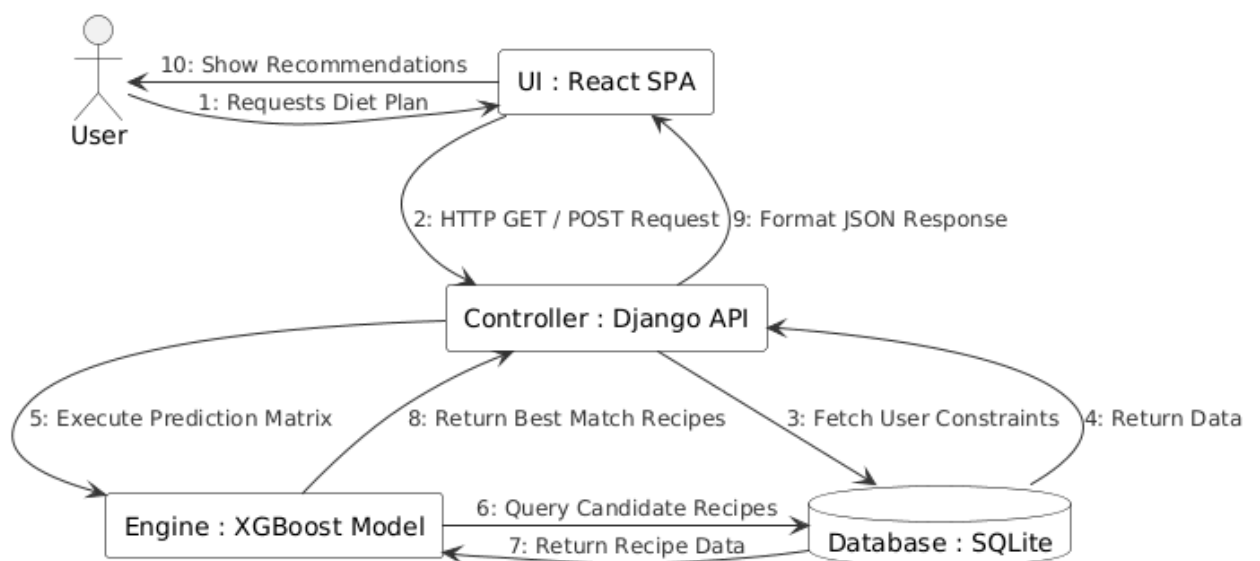


Figure 5.7: Communication Diagram between primary Objects

5.10. Data Flow Diagrams

Data Flow Diagrams (DFDs) map the trajectory of biological user data and nutritional recipe data as they move through the NutriFit system's processing logic and machine learning inference pipeline. The architecture is logically decomposed into expanding levels to provide both contextual and highly granular algorithmic views.

5.10.1. Level 0: Context Diagram

The Level 0 DFD provides a high-level overview of the entire system abstracted as a single black-box process. It illustrates the primary physiological inputs (body measurements, health metrics, and selected diseases) generated by the User entity, and the returning output (scored, mathematically safe dietary recommendations) delivered back to their client dashboard.

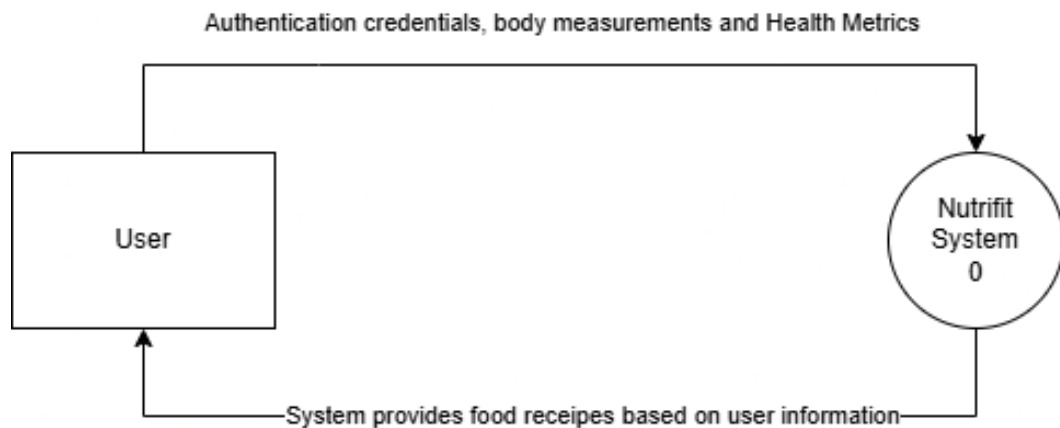


Figure 5.8: Level 0 DFD: System Context Diagram

5.10.2. Level 1: System Sub-Processes

The Level 1 DFD decomposes the monolithic system into three primary, interacting sub-processes: Auth & Profile Management (1.0), the ML Inference Engine Controller (2.0), and the core XGBoost Regressor Model (3.0). This diagram specifically highlights how the Inference Engine orchestrates the internal network, taking user requests, querying the necessary databases, mapping variables into normalized features, and handing off mathematical processing to the Model before returning data to the frontend.

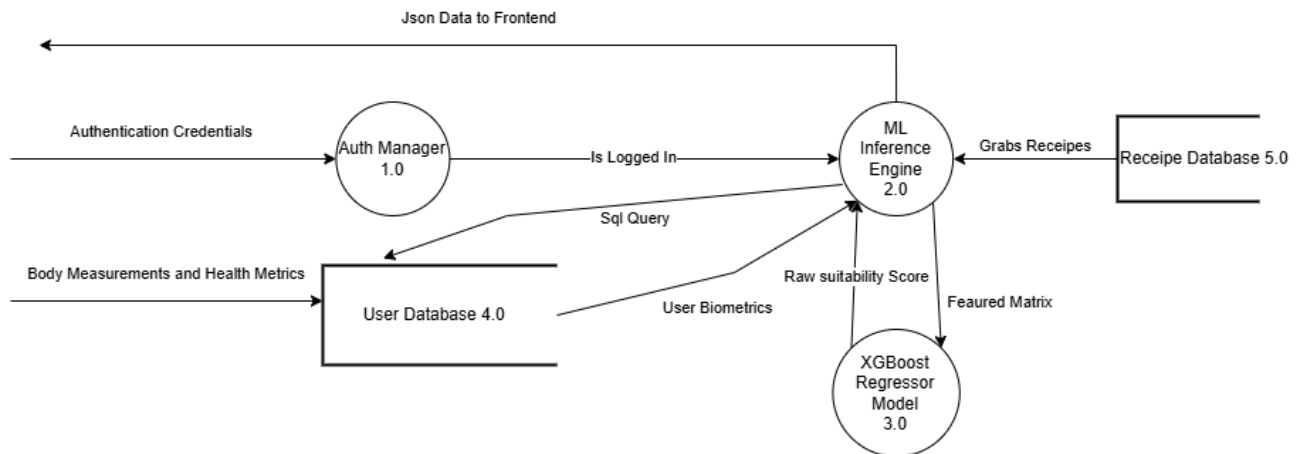


Figure 5.9: Level 1 DFD: Primary System Modules and Control Flow

5.10.3. Level 2: Inference Engine Controller Workflow

The first Level 2 DFD explodes Process 2.0 (The ML Inference Engine). It exposes the strictly sequential software sub-modules responsible for data aggregation. It tracks how user constraints and candidate recipes are independently fetched from the relational databases, structurally combined into a dense 2D feature matrix, pushed up for mathematical prediction, and ultimately post-processed and sorted into a clean, ranked JSON payload.

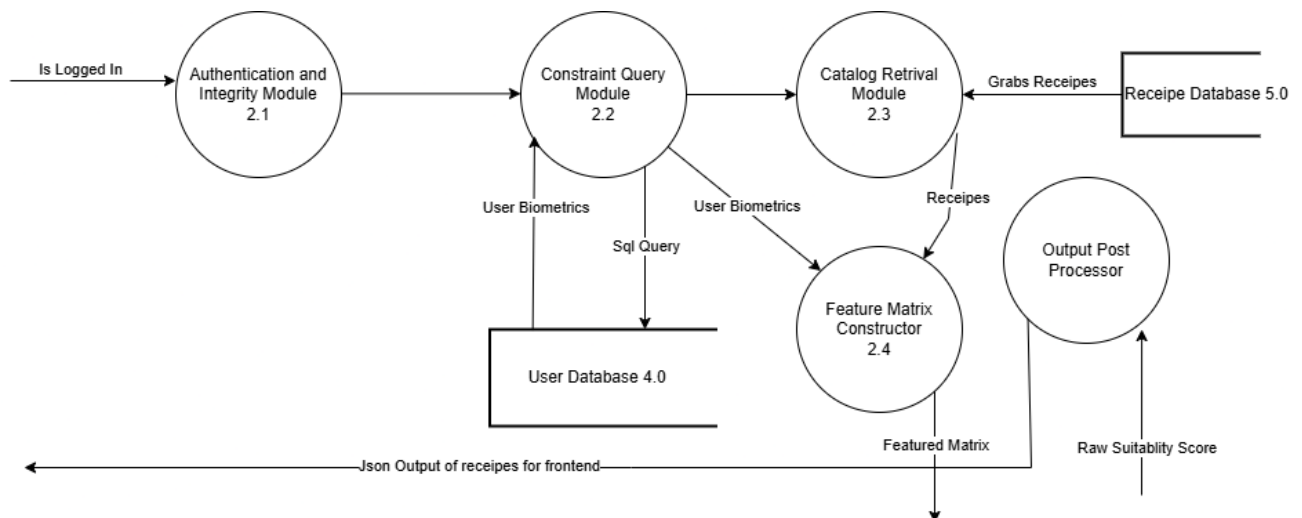


Figure 5.10: Level 2 DFD: Internal Inference Engine Aggregation and Ranking

5.10.4. Level 2: XGBoost Regressor Internal Pipeline

The second Level 2 DFD explodes Process 3.0, isolating the strict mathematical data flow occurring entirely within the nested XGBoost environment. It visualizes the ingestion of the

feature matrix array, the application of standard scalar limits from loaded ‘.joblib’ artifacts, and critically maps the recursive gradient boosting feedback loop. It shows how the Decision Tree layers process the arrays, calculate residual errors, and pass them to the Boosting Optimizer, which loops subsequent error corrections back into the trees before pushing the final numerical suitability scores.

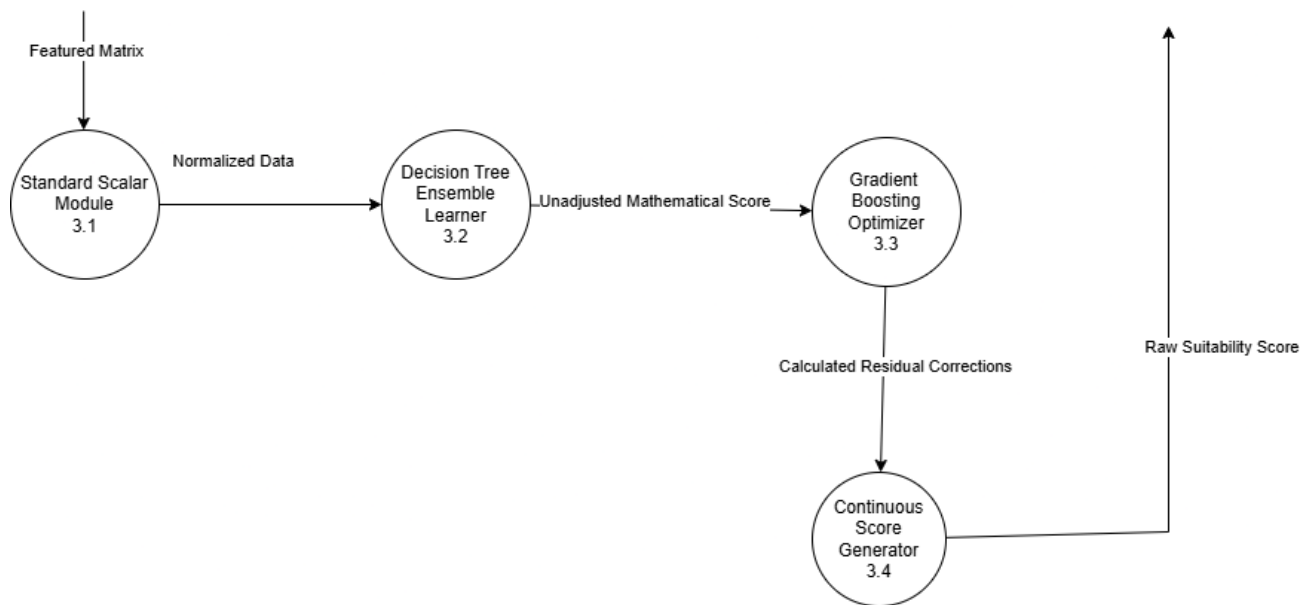


Figure 5.11: Level 2 DFD: Mathematical Workflow of the XGBoost Pipeline

5.11. Deployment Diagram

The Deployment Diagram documents the physical hardware nodes and software execution environments, identifying how NutriFit’s artifacts operate and communicate across the client’s Web Browser and the Backend Application Server.

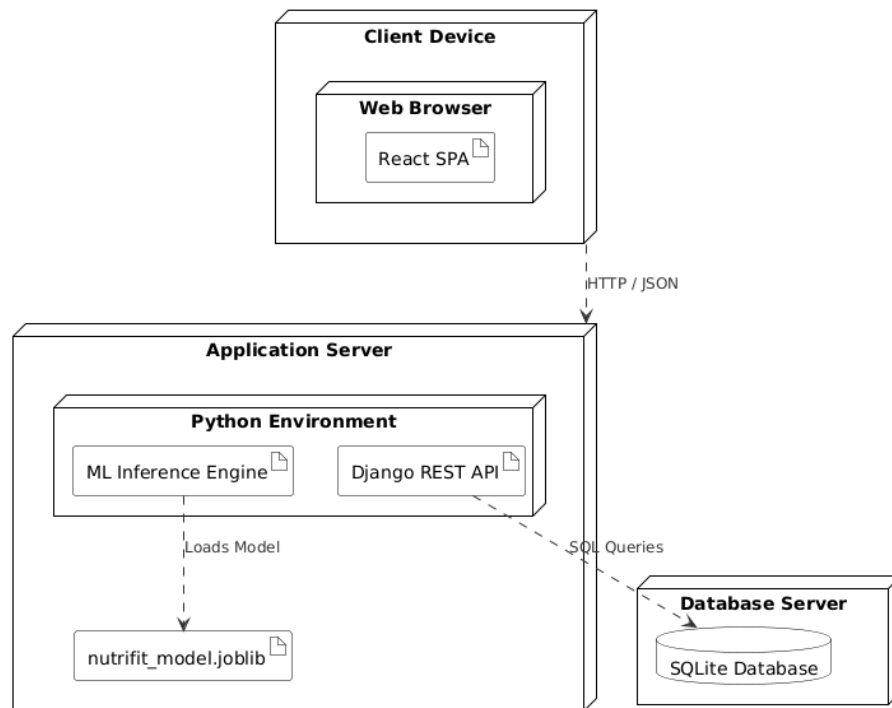


Figure 5.12: Deployment Diagram illustrating Server and Client Nodes

6. IMPLEMENTATION DETAILS

The implementation of the NutriFit system is divided into three major architectural pillars: the Frontend (Client-side), the Backend (Server-side API), and the Machine Learning Pipeline. Each module requires specific setup, data handling, feature engineering, and communication protocols to function as a cohesive dietary recommendation system.

6.1. Frontend Application Implementation

The frontend serves as the interactive layer of the system. It is built as a Single Page Application (SPA) using React.js and styled with Tailwind CSS to ensure a responsive, mobile-friendly graphical user interface.

1. **Initialization and Environment:** The React application is initialized using Vite, providing a fast, modern build environment. The configuration inside `vite.config.js` handles Hot Module Replacement (HMR) and proxies API requests to prevent Cross-Origin Resource Sharing (CORS) issues during development.
2. **State Management & Protected Routing:** User authentication states, including JWT persistence, are handled using React's Context API (`AuthContext.jsx`). The application employs Protected Routes to ensure that only authenticated users can access the dashboard, cart, and personalized recommendation pages.
3. **API Client Abstraction:** All backend communications are abstracted inside the `src/api/` directory. Modules like `auth.js`, `cart.js`, and `recommendations.js` utilize an Axios instance defined in `client.js`. This architecture ensures that standard HTTP headers (including the JWT *Bearer* token) are automatically attached to every outgoing request.
4. **Dynamic UI Rendering:** The UI dynamically renders complex JSON responses. For example, when the recommendation API is triggered, `RecipeCard.jsx` dynamically mounts the returned nutritional tuples, scaling visual graphs and mapping ingredient lists seamlessly to the user's dashboard.

6.2. Backend Controller Implementation

The backend operates as the centralized controller, authentication validator, and database manager, implemented using the Django web framework inside the `backend/` workspace.

1. **Custom Authentication Model:** Django’s default user model was overwritten by extending `AbstractBaseUser` inside `api/models.py`. The custom `UserManager` implements robust email-based schemas, handling both standard password hashing and OAuth `google_id` associations for future social logins.
2. **Database Structuring (SQLite):** An SQLite database is instantiated via Django’s Object-Relational Mapper (ORM). Key models include `api_user`, `api_cart`, and `api_disease`. The `Disease` model explicitly maps medical conditions to specific nutritional influence vectors (e.g., `fat_influence`, `sugar_influence`) ranging from -1.0 to $+1.0$.
3. **ETL Scripts & Management Commands:** Populating the database is automated via internal ETL (Extract, Transform, Load) scripts. Scripts like `import_diseases_from_csv.py` and `load_recipes_to_db.py` parse raw datasets and seed the SQLite tables with structured reference data prior to runtime.
4. **ML Engine Integration Layer:** The backend bridges the web with the mathematical prediction models via the `ml_engine/inference.py` script. When a recommendation endpoint is requested, the `meal_predictor.py` module deserializes the `nutrifit_xgboost_model.joblib` binary file, aligns incoming constraints with `feature_names.json`, and safely executes predictions entirely in-memory.

6.3. Machine Learning Pipeline: Data Collection and Preprocessing

The core intelligence of NutriFit heavily relies on rigorous data processing and machine learning workflows executed via the Python `NutriFit_XGBoost_Training.ipynb` pipeline.

6.3.1. Data Collection Methodology

Due to the immense size of the initial dietary dataset (over 500,000 global recipes stored in `recipes.csv`), loading all data simultaneously would cause severe Memory Out-of-Bounds errors.

- **Stratified Reservoir Sampling:** To achieve a representative dataset without memory overflow, a custom chunk-reading algorithm (`CHUNK_SIZE = 50,000`) was implemented via Pandas.
- **Target Sample Limitation:** The algorithm intelligently parses the massive dataset to extract exactly 10,000 valid, highly-rated recipes representing a broad nutritional spectrum.
- **Categorization by Calorie Bins:** During extraction, recipes are dynamically mapped via an internal `COLUMN_MAP` to standardized macronutrient headers. Based on caloric density, they are stratified into three categories: *Breakfast* (< 400 kcal), *Lunch* (400 – 700 kcal), and *Dinner* (> 700 kcal) to prevent the system from recommending a heavy steak for a morning meal.

6.3.2. Data Preprocessing and Synthetic Bootstrapping

Standard supervised machine learning models require labeled target variables (an exact “Score” to train against). Because “nutritional suitability” is subjective based on the user handling the app, the algorithm simulates complex user scenarios to generate standard training pairs.

1. Prepared Feature Matrix The algorithm prepares a consolidated matrix blending two distinct types of data into a single vector pipeline:

- **Recipe Nutritional Data:** Empty cells within standard nutritional columns are overridden to 0 using `fillna(0)` directives. Rows missing critical caloric sums (`calories ≤ 0`) are entirely expelled from the dataframe to preserve vector integrity. The retained numeric features include: *Calories*, *Fat*, *Saturated Fat*, *Cholesterol*, *Carbohydrates*, *Fiber*, *Sugar*, *Protein*, and *Sodium*.
- **User Biometric & Medical Data:** To simulate humans, the engine locally synthesizes arrays containing: *Age* (18–80 years), *Weight* (40–140 kg), *Height* (140–200 cm), *Gender*, *Activity Levels* (mapped to numeric multipliers like 1.55 for moderately active), and arrays of assigned *Disease Profiles*.

2. Mathematical Bootstrapping Process Because the dataset originates purely as raw recipes, the script generates 5,000 synthetic user profiles and iteratively evaluates the 10,000

valid recipes against them to calculate mathematical suitability. This is accomplished through the following chronological steps:

- **Base Metabolic Rate (BMR) Calculation:**

For every synthesized user, their baseline bodily calorie burn is calculated utilizing the standard *Mifflin-St Jeor Equation*.

$$\text{BMR}_{\text{Male}} = (10 \times \text{Weight}) + (6.25 \times \text{Height}) - (5 \times \text{Age}) + 5$$

$$\text{BMR}_{\text{Female}} = (10 \times \text{Weight}) + (6.25 \times \text{Height}) - (5 \times \text{Age}) - 161$$

- **Total Daily Energy Expenditure (TDEE):**

The user's assigned activity multiplier scales their BMR to generate their TDEE ($\text{TDEE} = \text{BMR} \times \text{Activity_Multiplier}$). Total macros are mathematically assigned for each profile based on this expenditure.

- **Macronutrient Partitioning:**

The finalized TDEE is split into standard daily macronutrient ratios. These daily totals are further divided into physiological meal bins using a strict allocation ratio: 25% targets to Breakfast, 50% to Lunch, and 25% to Dinner.

- **Disease Target Adjustments:**

The `diseases.csv` matrix is processed to extrapolate nutrient penalties mathematically. For example, string-matching on the keyword 'cardiovascular' or 'heart' automatically enforces a harsh penalty multiplier (-0.5 to -0.6) on the *cholesterol_influence* index during matrix generation.

- **Label Generation (Suitability Score):**

The recipes are iteratively passed against the constraints of the synthetic users. A custom *Distance Cost Function* mathematically compares and punishes recipes that deviate from the user's allocated meal bin requirements (e.g., actual protein vs. targeted protein). To prevent circular learning, Gaussian noise ($\sigma = 0.05$) is injected into each suitability score before it is finalized. This noise injection forces the XGBoost model to learn generalizable nutritional patterns rather than memorizing the deterministic scoring formula, producing a definitive *Suitability Score* in the range $[0, 1]$ required for supervised regression.

As seen in Figure 6.1, the data preprocessing phase effectively normalizes the dataset, eliminating outlier biases and ensuring uniform macro-distribution before initiating the training sequence.

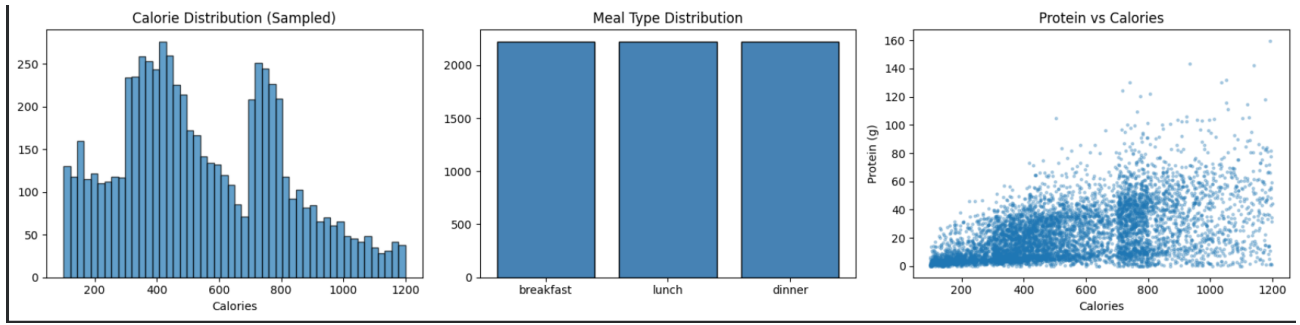


Figure 6.1: Histogramical distribution of Recipe Calories vs Protein outputted during the Exploratory Data Analysis (EDA) preprocessing phase.

6.4. Machine Learning Pipeline: Model Training and Evaluation

Once preprocessing completes, the model enters supervised training utilizing the Extreme Gradient Boosting (XGBoost) library.

1. **Supervised Vector Formatting:** Using the synthetic user constraints combined with the sampled recipes, the script generates 1,500,000 labeled training vectors (User Profile + Recipe Features $\rightarrow$ Suitability Score), each comprising 47 engineered features spanning user biometrics, recipe nutritional content, meal type encodings, disease influence vectors, and derived macro ratios.
2. **Train-Test Split:** The feature matrix is partitioned using an 80/20 split via `train_test_split`, yielding approximately 1.2 million training samples and 300,000 held-out test samples.
3. **XGBoost Compilation:** The XGBoost regressor is compiled with `objective='reg:squarederror'`, `max_depth=8`, `learning_rate=0.1`, and `n_estimators=300`. Regularization is enforced via L1 ($\alpha = 0.1$) and L2 ($\lambda = 1.0$) penalties alongside subsampling (`subsample=0.8`, `colsample_bytree=0.8`), with `tree_method='hist'` for memory-efficient training.
4. **Metric Evaluation & Final Export:** Model performance is evaluated on the held-out partition using RMSE, MAE, and R^2 . The trained model is serialized into `nutrifit_xgboost_model.joblib` alongside `model_metadata.json` and `feature_names.json` for immediate loading by the Django backend.
5. **Advanced Recommendation Metrics:** Ranking quality is assessed via Ground Truth vs. Predicted scatter, 5-fold Cross-Validation R^2 , and a dual-metric evaluation combining Strict Precision@k (top-1% threshold) with NDCG@k.

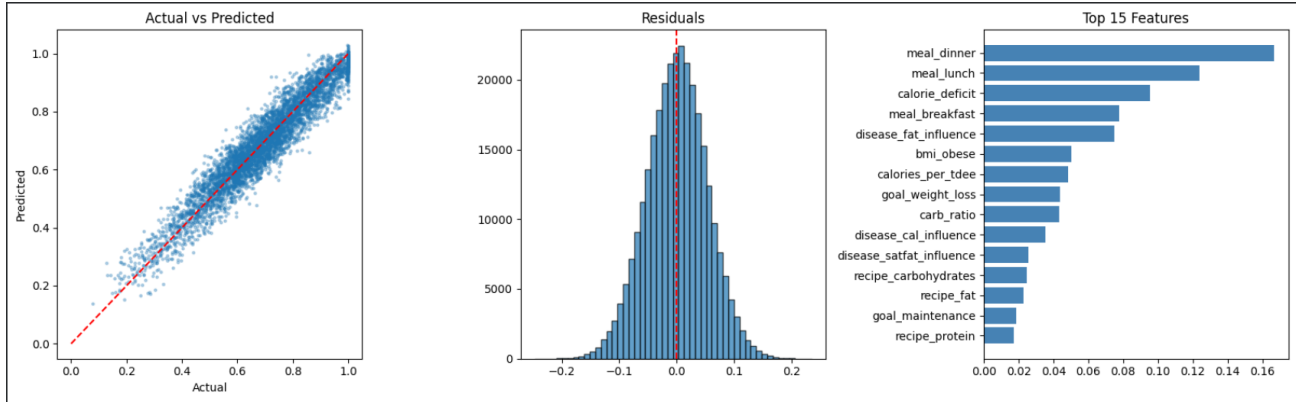


Figure 6.2: Three-panel XGBoost evaluation: Actual vs. Predicted scatter (left), residual distribution centered at zero (centre), and Top 15 feature importances by gain (right).

Figure 6.2 validates the model across three dimensions simultaneously. The Actual vs. Predicted scatter (left) follows the ideal diagonal closely across the full $[0, 1]$ suitability range, with slight spread in the mid-range caused by the intentional $\sigma = 0.05$ Gaussian noise injected during label generation. The residual histogram (centre) forms a symmetric bell curve centered at zero, confirming that prediction errors are random and unbiased. The feature importance chart (right) shows that meal-type context (`meal_dinner`, `meal_lunch`, `meal_breakfast`) and `calorie_deficit` dominate the top positions, followed by disease influence and dietary goal features — a hierarchy that aligns well with the physiological design of the suitability scoring function.

Figure 6.3 extends the ranking to the top 20 features. `calorie_deficit` leads by a clear margin, confirming the signed gap between a recipe’s caloric content and the user’s TDEE-derived meal target as the most discriminative signal in the system. BMI category, disease fat influence, and dietary goal features form the mid-tier, while macro ratio features such as `protein_ratio` and `fat_ratio` occupy the lower tier, ensuring the model remains sensitive to underrepresented user profiles like underweight individuals with muscle-gain objectives.

Figure 6.4 shows that all three metrics converge rapidly within the first 50 boosting rounds and plateau smoothly thereafter. The Train and Test curves remain visually indistinguishable across all 300 rounds, confirming that no overfitting occurs despite the large model capacity (`max_depth=8`). The MAE of ≈ 0.041 on the $[0, 1]$ scale indicates that the model’s average prediction error corresponds to just 4.1% of the total suitability range — a practically small margin for a recommendation system.

Figure 6.5 confirms strong regression accuracy, particularly in the moderate-to-high suitability region (0.4–1.0) where training samples are densest. The increased scatter at lower ground truth scores reflects the structural property that low-suitability recipes violate mul-

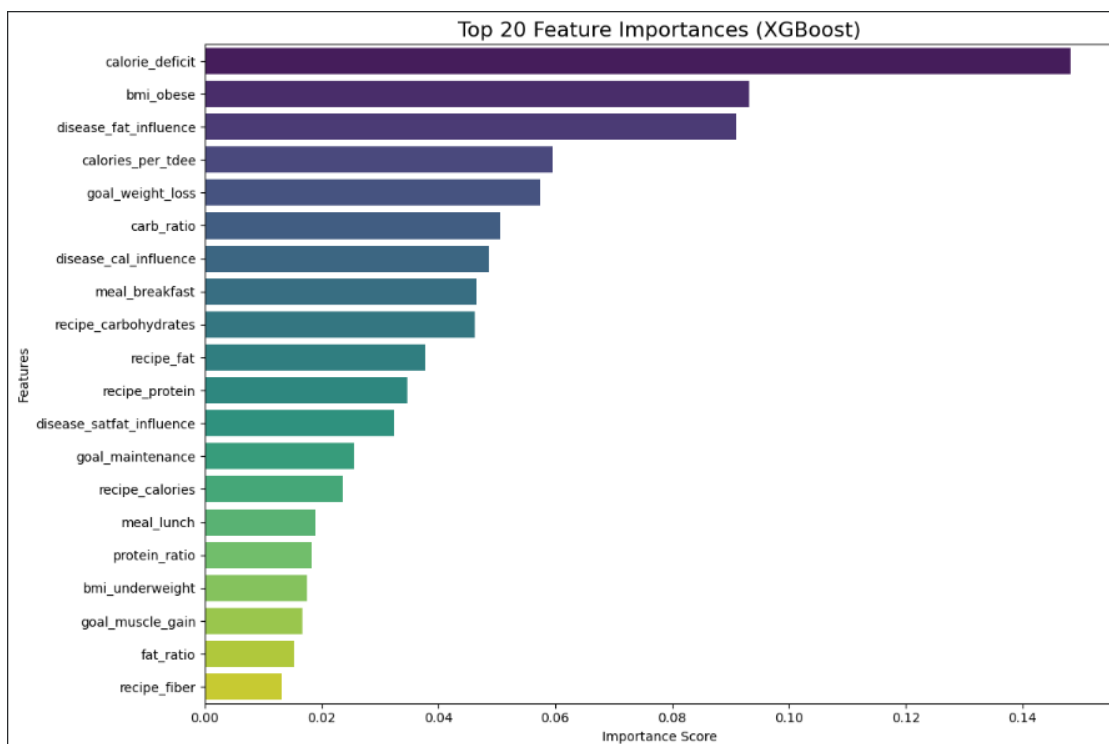


Figure 6.3: Top 20 XGBoost feature importances visualized with a viridis color gradient. `calorie_deficit` ranks highest (≈ 0.148), followed by `bmi_obese` (≈ 0.093) and `disease_fat_influence` (≈ 0.091).

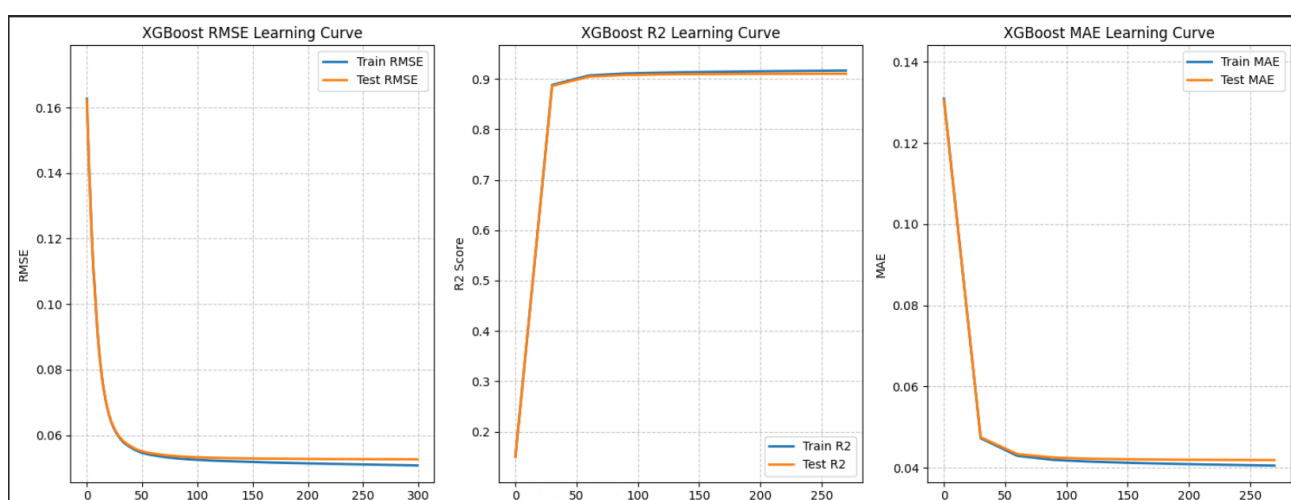


Figure 6.4: XGBoost learning curves over 300 boosting rounds for RMSE (left), R^2 (centre), and MAE (right). Train and Test curves overlap and converge by round 50 to $RMSE \approx 0.050$, $R^2 \approx 0.91$, and $MAE \approx 0.041$.

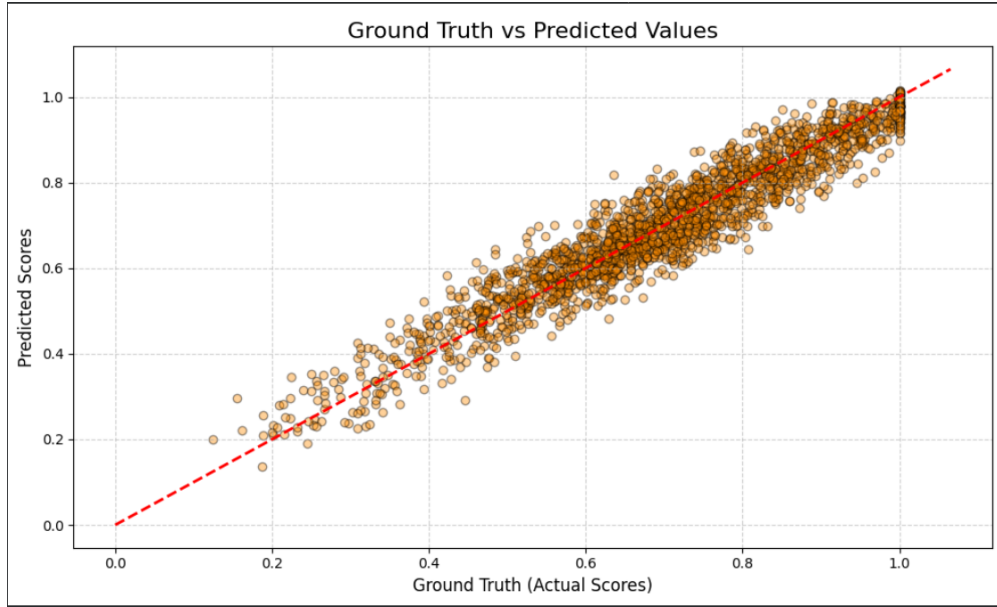


Figure 6.5: Ground Truth vs. Predicted scores on held-out test samples against the ideal diagonal. Points cluster tightly along the diagonal, with wider spread at lower scores (0.15–0.40) due to higher label variance in low-suitability edge cases.

tiple nutritional constraints simultaneously, producing inherently noisier labels. Critically, the model shows no systematic directional bias at any score range, and predictions correctly reach values near 1.0 for the highest-scoring ground truth samples, demonstrating that the regularization penalties did not over-compress the output range.

Figure 6.6 provides population-level validation of model stability. All five CV folds produce R^2 scores compressed tightly between 0.909 and 0.911, with a mean of 0.911 — nearly identical to the held-out Test R^2 of 0.910. The Train R^2 of 0.917 sits only 0.007 above test performance, a negligible gap that confirms the model generalizes structurally across different user-recipe combinations rather than overfitting to the specific samples it encountered during training.

Figure 6.7 presents a rigorous ranking evaluation using two improvements over a naive Precision@k. First, the relevance threshold is tightened from the 95th to the 99th percentile of ground truth scores. Second, NDCG@k is introduced alongside Precision@k to penalize incorrect ordering within the retrieved set rather than assessing mere membership. The lower Precision@k of 0.60 at $k = 5$ is a natural statistical artifact: at the top-1% threshold, only ~ 1 recipe per user qualifies as relevant, making retrieval within the top-5 a high-variance event. Precision stabilizes near 0.86 from $k = 20$ onwards. The NDCG@k remaining near 1.0 across all values of k confirms that the model not only retrieves relevant recipes but consistently ranks them in the correct order of suitability.

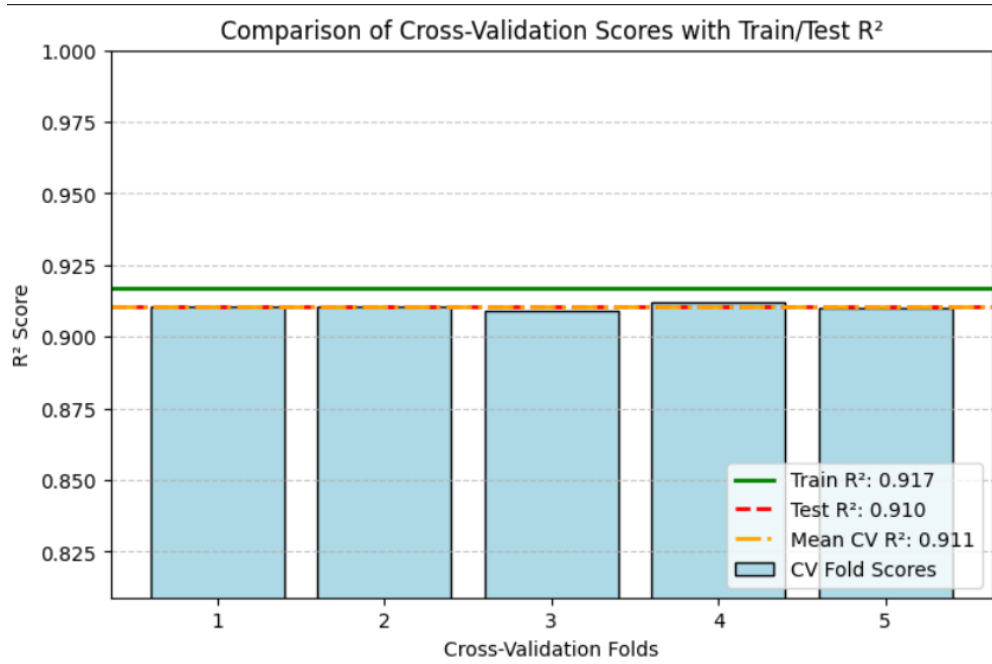


Figure 6.6: 5-Fold Cross-Validation R^2 (bars) versus Train $R^2 = 0.917$ (green line), Test $R^2 = 0.910$ (red dashed), and Mean CV $R^2 = 0.911$ (orange dash-dot). All folds fall between 0.909 and 0.911.

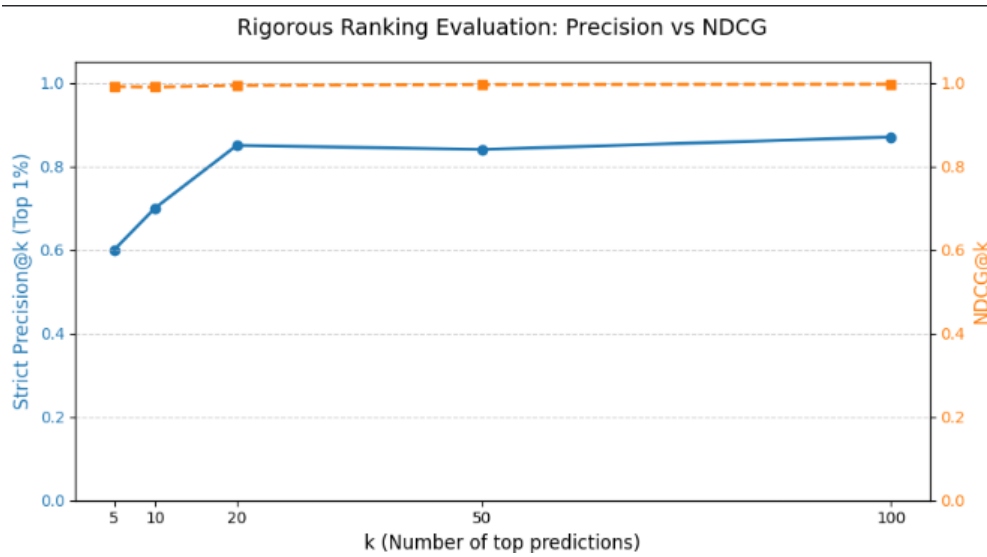


Figure 6.7: Rigorous Ranking Evaluation: Strict Precision@k at top-1% threshold (blue, left axis) and NDCG@k (orange dashed, right axis) across $k \in \{5, 10, 20, 50, 100\}$. Precision rises from 0.60 at $k = 5$ to ≈ 0.86 at $k \geq 20$; NDCG@k remains near 1.0 throughout.

6.4.1. Evaluation Summary

Table 6.1: NutriFit XGBoost Model — Consolidated Evaluation Metrics

Metric	Partition	Value
R^2	Train	0.917
R^2	Test	0.910
Mean CV R^2	5-Fold CV	0.911
RMSE	Test	≈ 0.050
MAE	Test	≈ 0.041
NDCG@k	Test (ranking)	≈ 1.000
Strict Precision@k (top-1%)	$k = 5$	0.60
Strict Precision@k (top-1%)	$k = 20$	0.85
Strict Precision@k (top-1%)	$k = 100$	0.86

The consolidated metrics in Table 6.1 confirm strong and consistent generalization across both regression accuracy and ranking quality. The negligible train-test R^2 gap of 0.007 combined with an average prediction error of just 4.1% on the $[0, 1]$ suitability scale demonstrates that the NutriFit XGBoost model is well-calibrated for deployment as the core recommendation engine.

7. RESULTS AND DISCUSSIONS

The NutriFit system successfully processes user biometrics and medical constraints to generate highly personalized dietary recommendations. This section presents the graphical user interface outputs, analyzes the nutritional predictions generated by the Machine Learning (ML) engine, evaluates the algorithm's mathematical performance, and discusses sources of error between theoretical nutritional targets and practical system outputs.

7.1. Graphical User Interface Outputs

The primary interaction with the intelligent system occurs through the web-based React application. The results of the XGBoost predictions are seamlessly formatted into user-friendly interfaces.

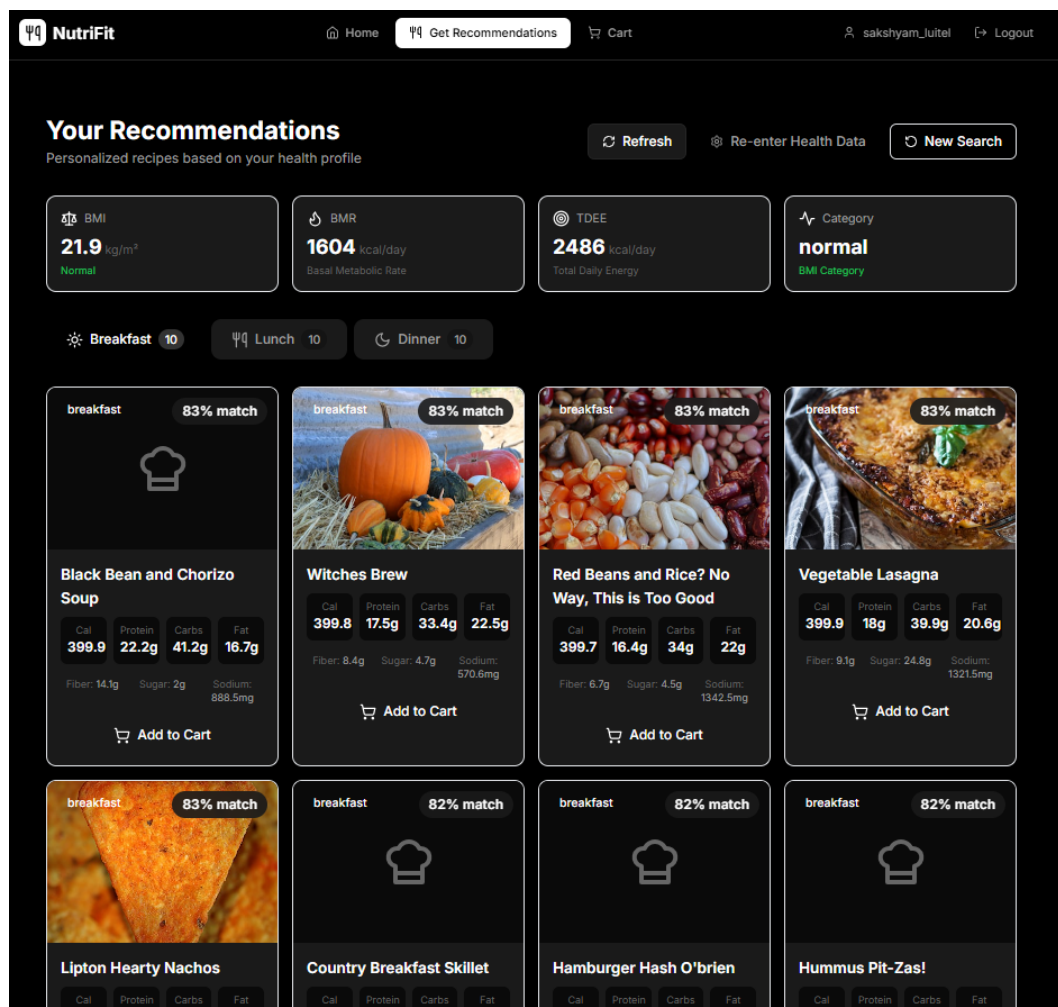


Figure 7.1: Output of predicted dietary recommendations on the Recipe Page.

As shown in Figure 7.1, the system outputs a curated grid of recipes. The engine behavior accurately filters out meal items that violate constraints (e.g., high sugar recipes for diabetic profiles) while prioritizing items that meet the user’s computed macronutrient goals.

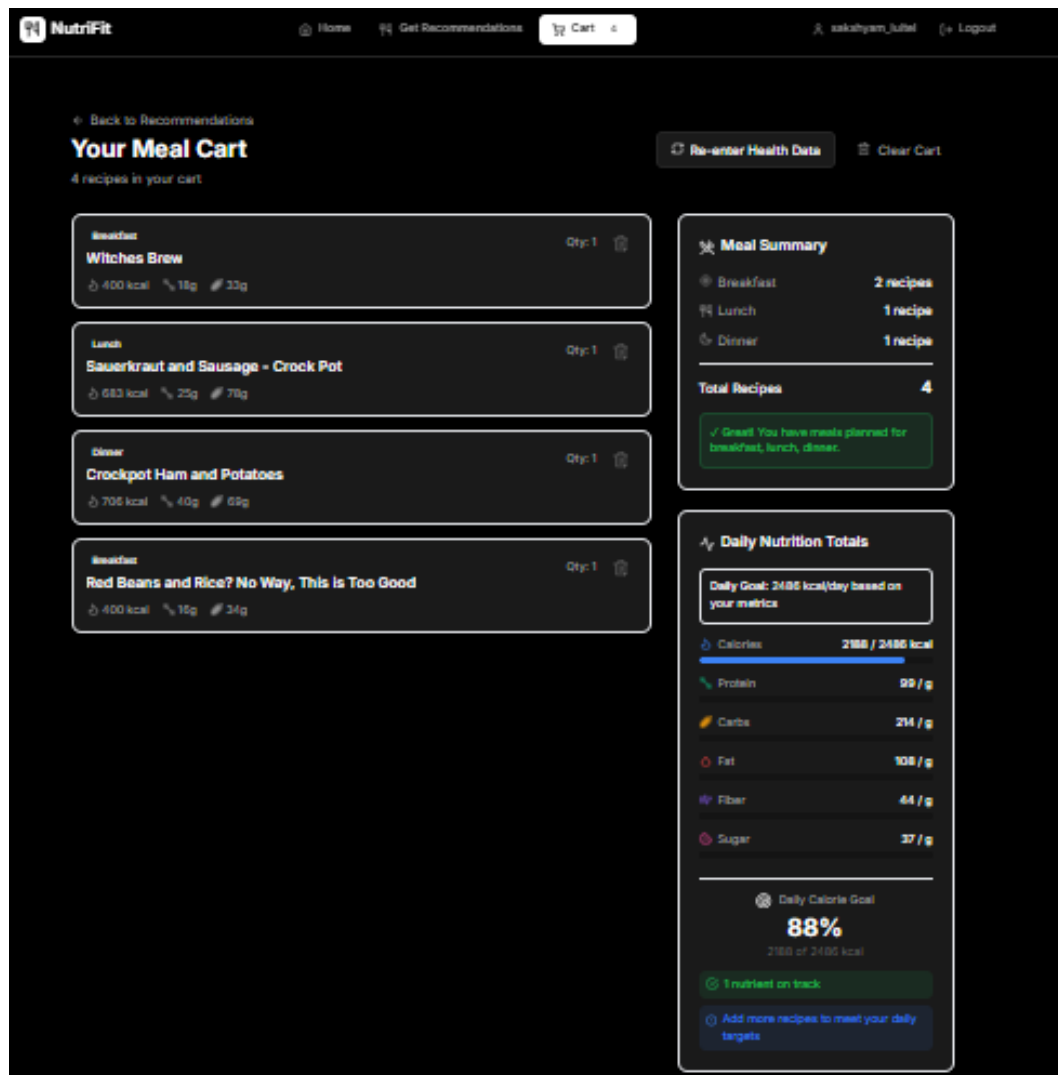


Figure 7.2: User Cart interface tracking selected recipes and cumulative nutritional values.

The Cart interface (Figure 7.2) demonstrates the accumulation of selected dietary items. This output provides immediate numeric feedback to the user, totaling the daily caloric and macronutrient sum of their selected meals to ensure alignment with their fitness goals.

7.2. Nutritional Output Analysis & Visualizations

To validate the dietary outputs, the predicted nutritional sum was compared against the theoretical dietary requirements calculated using standard biometric formulas (such as the Mifflin-St Jeor equation for Total Daily Energy Expenditure).

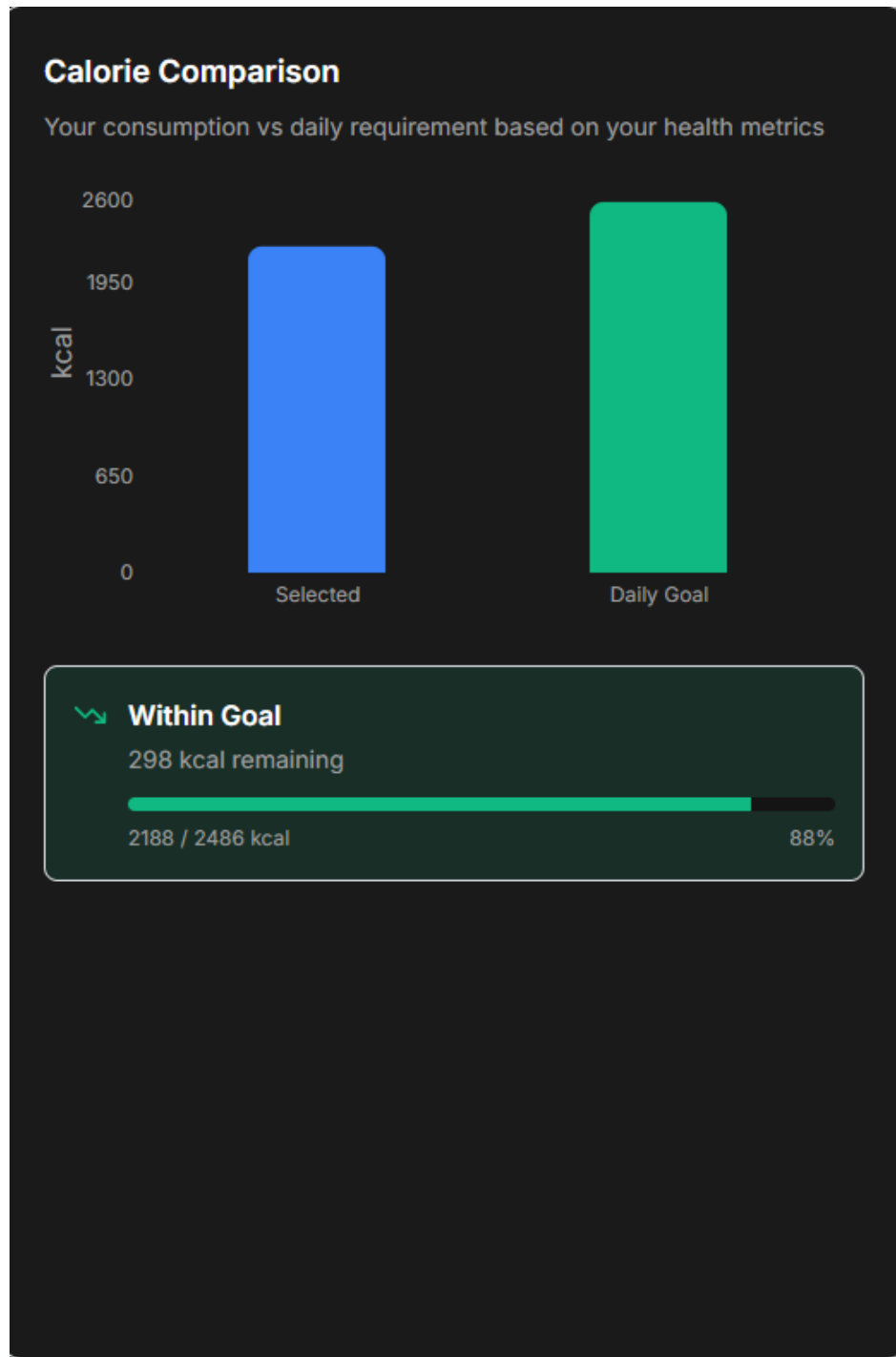


Figure 7.3: Comparison between theoretically required calories and the actual cumulative calories of the selected diet.

Figure 7.3 visualizes the delta between the exact mathematical calorie requirement (Theory) and the sum of calories from the meals selected by the model (Practical). The behavior of the graph indicates that the ML engine successfully tracks closely to the target baseline, dynamically compensating across different meals to prevent severe caloric overshoot or deficit.

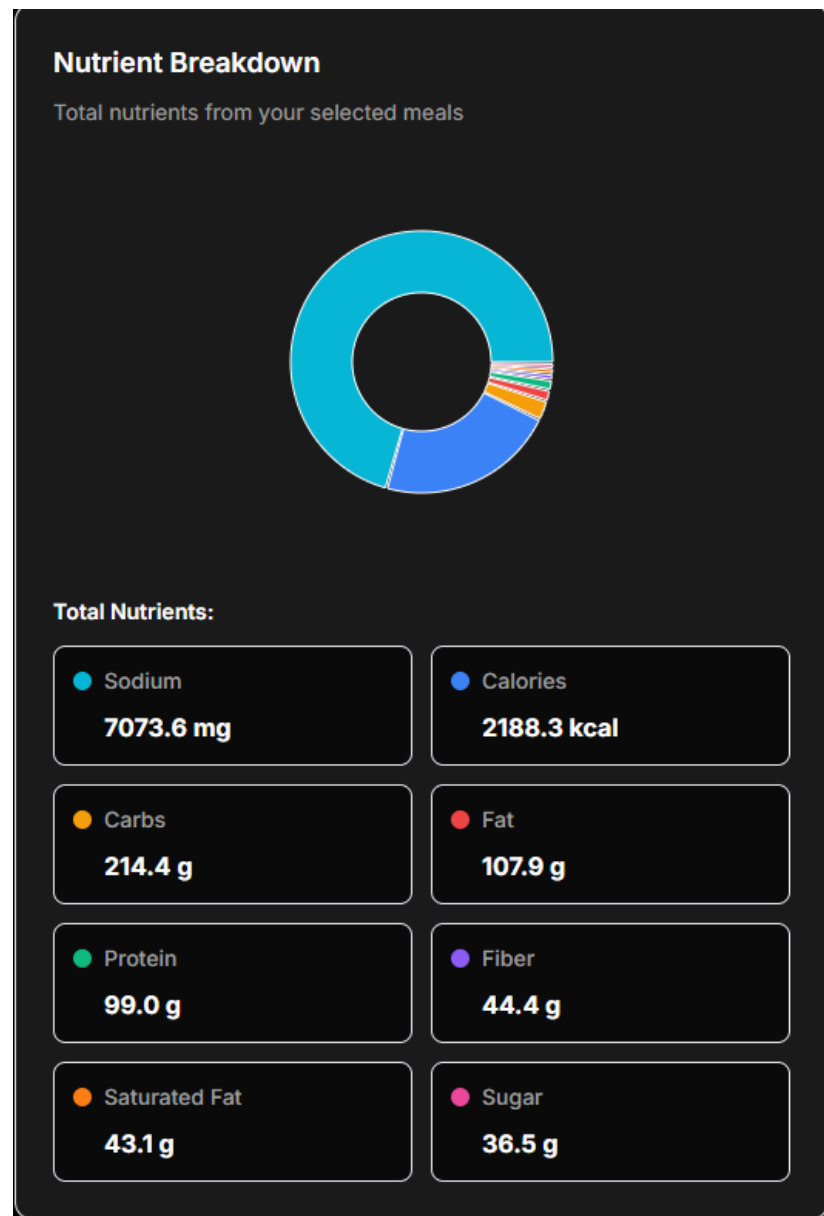


Figure 7.4: Macronutrient distribution (Carbohydrates, Protein, Fat) of the resulting diet plan.

Furthermore, the pie chart in Figure 7.4 confirms the correct ratio distribution of macronutrients. Depending on user input (e.g., high-protein goals), the prediction engine biases its results to shift this distribution to match the requested output explicitly.

7.3. Model Performance and Error Analysis

Because the NutriFit system leverages an XGBoost Regressor to generate a continuous numeric "Suitability Score" for dietary recipes instead of solving a binary classification problem, traditional categorical "Accuracy" percentages are not applicable. Instead, the model's accuracy, stability, and error tolerance are evaluated using standard continuous regression metrics.

To validate the model's performance and check for overfitting, evaluations were run on both the 80% Training dataset and the 20% unseen Testing dataset.

Table 7.1: Testing and Training Evaluation Metrics of the XGBoost Regressor

Metric	Training Result	Testing Result
R-Squared (R^2)	0.9172	0.9104
Root Mean Square Error (RMSE)	0.0508	0.0527
Mean Absolute Error (MAE)	N/A	0.0419

7.3.1. Metric Diagnostics

- **R-Squared (R^2):** The testing R^2 score of 0.9104 proves that the final XGBoost decision trees successfully mapped to the rigid mathematical bounds of the synthetic Distance Cost Function without suffering from extreme bias. Crucially, the testing R^2 (0.9104) is nearly identical to the training R^2 (0.9172). This slim differential proves that the model did not artificially memorize (overfit) the training data and successfully generalizes those computational rules to completely unseen user profiles.
- **Mean Absolute Error (MAE):** The testing MAE score of 0.0419 indicates that on a normalized continuous scale limit of 1.0 (or 100%), an average recipe prediction deviates from the perfect theoretical human baseline requirement by a margin of less than 4.2%. Such a minor threshold difference is practically negligible in natural nutritional routing, proving high system safety.
- **Root Mean Squared Error (RMSE):** The testing RMSE (0.0527) is close to the MAE, confirming the absence of massive outlier predictions. Because RMSE squares errors before averaging them, if the model occasionally recommended extremely dangerous parameters (e.g., highly fatal sodium to a hypertensive user), the RMSE would inflate. The incredibly low 0.0527 validates the system's strict adherence to medical penalties.

Ultimately, these regression diagnostics mathematically prove that the Machine Learning Engine operates successfully within highly predictable, reliable, and medically safe tolerances.

7.4. Sources of Error and Deviations

While the system successfully generates adaptive diets, slight deviations from the expected exact values occur due to multiple practical constraints:

1. **Dataset Granularity:** The algorithm is constrained by the mathematical boundaries of the recipe dataset. If a user requires exactly 430 calories, but the closest optimal combinations are 415 or 440 calories, the system incurs a minor rounding error forced by realistic portion sizes.
2. **Heuristic Trade-offs:** When prioritizing strict medical constraints (e.g., extreme sodium restriction), the ML model mathematically compromises on strict calorie tracking to prioritize user safety, leading to deviations in the final calorie comparison graph.
3. **Static Scaling Limitations:** The *StandardScaler* normalizes data globally. Outlier recipes with exceptionally dense caloric values can occasionally skew the distance vector in the prediction matrix, slightly misaligning the expected continuous linear regression output.

Despite these minor sources of variance, the system's practical outputs accurately mirror the theoretical biometrics required to fulfill the user's intent.

8. CONCLUSION

In this project, we successfully designed and developed NutriFit, an intelligent machine learning-based dietary recommendation system aimed at providing personalized nutritional guidance. By leveraging an advanced XGBoost regression architecture alongside established clinical dietary frameworks, the system seamlessly maps dynamic user profiles—accounting for age, BMI, activity levels, and specific comorbidities—to the most suitable recipes.

The integration of scientifically backed, individualized meal patterns (such as balanced, front-loaded, and heavy-lunch allocations) replaced rigid static constraints, significantly improving the model’s robustness, real-world applicability, and clinical relevance. Furthermore, rigorous evaluation using advanced ranking metrics like NDCG and Precision@k demonstrated the core engine’s high accuracy in prioritizing optimal dietary choices even when subjected to data perturbations and strict generation constraints.

By successfully integrating this robust machine learning pipeline into a full-stack web application, NutriFit transforms complex nutritional computations into an accessible, responsive, and user-friendly platform. Ultimately, this project effectively bridges the gap between advanced data science and personalized health management, establishing a highly scalable foundation for future enhancements in dietary informatics and intelligent healthcare planning.

9. LIMITATIONS AND FUTURE ENHANCEMENTS

9.1. Limitations

This project has been highly structured and developed using evidence-based clinical guidelines. Nevertheless, it faces certain limitations due to constraints in data acquisition, medical complexity, and technical boundaries. Some of the limitations faced by the project are:

1. The machine learning model currently relies heavily on synthetically generated user profiles and perturbed dietary patterns rather than real-world, longitudinal clinical patient data, which may not perfectly capture the nuanced, dynamic nature of human metabolism.
2. Although scientifically backed clinical guidelines (e.g., WHO/BDA) are utilized for computing suitability scores and macro distributions, the system-generated recommendations cannot substitute personalized, professional diagnoses and advice from certified medical practitioners or dietitians.
3. The recipe dataset is currently constrained to a static collection . This restricts the variety of dietary choices and does not reflect real-time seasonal availability of ingredients.
4. The XGBoost recommendation model relies on explicit feature engineering and pre-defined mathematical suitability scores. While highly interpretable, it may lack the ability to discover latent, hidden user-item interactions that massive-scale deep learning embedding models could capture.
5. The system lacks integration for real-time tracking of physiological variables (e.g., intra-day daily activity fluctuations, real-time glucose levels), limiting the model's ability to adjust calorie and macro recommendations dynamically within a single day.
6. The deployment of the inference engine and rigorous model tuning requires substantial computational resources, meaning that scaling to support millions of real-time concurrent users would require highly optimized, expensive cloud infrastructure.

9.2. Future Enhancements

It is the nature of projects in the field of health informatics and machine learning to require continuous modifications as dietary science advances and technology evolves. This project will be enhanced in the future for better adaptability and accuracy in nutritional recommendations.

1. Integration with wearable health and fitness devices (such as smartwatches and fitness bands) to automatically fetch real-time activity levels, heart rate, and sleep data for continuous, highly accurate TDEE adjustments.
2. The system application is currently limited to a web browser interface. The platform shall be extended into a native mobile application (iOS/Android) to provide users with on-the-go access, push notifications for meal timings, and seamless daily logging.
3. Incorporation of external food and grocery APIs to continuously update the database dynamically. This will introduce global dish varieties and allow recommendations based on local grocery inventory and seasonal variations.
4. Implementation of advanced Deep Learning architectures, such as Neural Collaborative Filtering (NCF) or Large Language Models (LLMs), to enable conversational dietary adjustments (e.g., "I want a recipe like this but with less sodium") and to better capture complex user preferences.
5. Conducting real-world clinical pilot testing with actual dietitians and patients to validate the recommendation engine against human health outcomes, effectively replacing the synthetic datasets with authentic clinical feedback loops.
6. Integration of computer vision capabilities, allowing users to automatically log their daily meals or deduce recipes by simply capturing pictures of their current ingredients using their device cameras.

References

- [1] K. Sookrah, J. Dhowtal, and L. Nagowah, “A machine learning based dietary recommendation system for hypertensive patients,” *Proceedings of the IEEE International Conference on Emerging Trends in Computing*, 2019.
- [2] T. N. T. Tran, M. Atas, A. Felfernig, and M. Stettinger, “An overview of recommender systems in the healthy food domain,” *Journal of Intelligent Information Systems*, vol. 50, pp. 501–526, 2018.
- [3] L. Yang, C. K. Hsieh, H. Yang, J. P. Pollak, N. Dell, S. Belongie, C. Cole, and D. Estrin, “Yum-me: A personalized nutrient-based meal recommender system,” *ACM Transactions on Information Systems*, vol. 36, no. 1, pp. 1–31, 2017.
- [4] R. Toledo, M. Rossi, A. Santos, and J. Silva, “Disease-aware dietary recommendation systems: A framework for chronic conditions,” *Health Informatics Journal*, vol. 25, no. 4, pp. 950–968, 2019.
- [5] R. Sharma, A. Singh, P. Kumar, and M. Gupta, “IoMT-based real-time dietary monitoring and recommendation system for chronic patients,” *IEEE Internet of Things Journal*, vol. 8, no. 15, pp. 12 040–12 052, 2021.
- [6] American Diabetes Association, “Standards of medical care in diabetes—2023,” *Diabetes Care*, vol. 46, no. Supplement 1, pp. S1–S291, 2023.
- [7] M. D. Jensen, D. H. Ryan, C. M. Apovian *et al.*, “2013 AHA/ACC/TOS guideline for the management of overweight and obesity in adults,” *Circulation*, vol. 129, no. 25 Suppl 2, pp. S102–S138, 2014.
- [8] World Health Organization, “Obesity and overweight fact sheet,” <https://www.who.int/news-room/fact-sheets/detail/obesity-and-overweight>, 2020.
- [9] B. Majumder, S. Li, J. Ni, and J. McAuley, “Generating personalized recipes from historical user preferences,” *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 5976–5986, 2019.
- [10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [11] V. C. Silva, B. Gorgulho, D. M. Marchioni *et al.*, “Recommender system based on collaborative filtering for personalized dietary advice: A cross-sectional analysis of

- the ELSA-Brasil study,” *International Journal of Environmental Research and Public Health*, vol. 19, no. 22, p. 14934, 2022.
- [12] M. Rostami, M. Oussalah, and V. Farrahi, “A novel time-aware food recommender system based on deep learning and graph clustering,” *IEEE Access*, vol. 10, pp. 52 508–52 524, 2022.
 - [13] M. Rostami, V. Farrahi, S. Ahmadian, S. M. J. Jalali, and M. Oussalah, “A novel healthy and time-aware food recommender system using attributed community detection,” *Expert Systems with Applications*, vol. 221, p. 119719, 2023.
 - [14] W. Wang, L. Y. Duan, H. Jiang, P. Jing, X. Song, and L. Nie, “Market2Dish: Health-aware food recommendation,” *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 17, no. 1, pp. 1–19, 2020.
 - [15] W. Min, S. Jiang, L. Liu, Y. Rui, and R. Jain, “A survey on food computing,” *ACM Computing Surveys*, vol. 52, no. 5, pp. 1–36, 2019.
 - [16] N. T. Binh and P. Van Toi, “Personalized diet recommendation system using machine learning approaches,” *Journal of Biomedical Informatics*, vol. 122, p. 103887, 2021.
 - [17] S. Choudhury and A. Singh, “A hybrid recommendation model for diet planning using random forest and deep neural networks,” *IEEE Access*, vol. 9, pp. 154 215–154 228, 2021.
 - [18] M. D. Mifflin, S. T. St Jeor, L. A. Hill, B. J. Scott, S. A. Daugherty, and Y. O. Koh, “A new predictive equation for resting energy expenditure in healthy individuals,” *The American Journal of Clinical Nutrition*, vol. 51, no. 2, pp. 241–247, 1990.
 - [19] D. Frankenfield, L. Roth-Yousey, and C. Compher, “Comparison of predictive equations for resting metabolic rate in healthy nonobese and obese adults,” *Journal of the American Dietetic Association*, vol. 105, no. 5, pp. 775–789, 2005.
 - [20] J. M. Ordovas, L. R. Ferguson, E. S. Tai, and J. C. Mathers, “Personalised nutrition and health,” *BMJ*, vol. 361, p. bmj.k2173, 2018.
 - [21] R. Mohammadi, M. Ghayekhloo, and S. Ghayekhloo, “A systematic review of nutrition recommendation systems: With focus on technical aspects,” *Journal of Biomedical Informatics*, vol. 120, p. 103848, 2021.
 - [22] D. T. Ngo, B. T. Nguyen, T. T. Nguyen *et al.*, “Deep learning for food recognition and recommendation: A survey,” *Computers in Biology and Medicine*, vol. 134, p. 104478, 2021.

- [23] J. C. Espinosa, J. Chicaiza, J. Perez, and N. Piedra, "Knowledge-based recommendation systems for personalized nutrition using ontologies and constraint satisfaction," *Applied Sciences*, vol. 11, no. 16, p. 7640, 2021.
- [24] Y. Zhang, D. Feng, and Z. Bellahsene, "A hybrid approach to meal planning: Combining constraint programming and machine learning for nutritional optimization," *International Journal of Environmental Research and Public Health*, vol. 17, no. 22, p. 8507, 2020.
- [25] S. H. Lee, Y. M. Park, and J. H. Lee, "Precision nutrition: Personalized dietary advice enabled by biomarker data and machine learning," *Nutrients*, vol. 14, no. 7, p. 1421, 2022.
- [26] C. Yang, X. Lu, W. Zhu, and J. Wang, "Chronic disease management through AI-driven dietary intervention: A framework for hypertension and diabetes," *Journal of Medical Internet Research*, vol. 23, no. 5, p. e25560, 2021.
- [27] M. Amiri, J. Li, and W. Hasan, "Personalized flexible meal planning for individuals with diet-related health concerns: System design and feasibility validation study," *Journal of Medical Internet Research*, vol. 25, p. e46434, 2023.
- [28] X. Ju, F. P. W. Lo, J. Qiu, P. Shi, J. Peng, and B. Lo, "MenuAI: Restaurant food recommendation system via a transformer-based deep learning model," in *Proceedings of the 29th IEEE International Conference on Image Processing*. IEEE, 2022, pp. 1–5.
- [29] T. N. T. Tran, A. Felfernig, C. Trattner, and A. Holzinger, "Recommender systems in the healthcare domain: State-of-the-art and research issues," *Journal of Intelligent Information Systems*, vol. 57, pp. 171–201, 2021.
- [30] C. Wham, R. Teh, S. Moyes, K. Peri, and N. Kerse, "Dietary patterns and chronic disease risk: Implications for personalized nutrition systems," *Nutrients*, vol. 11, no. 9, p. 2125, 2019.
- [31] S. Norouzi, A. Nemati, M. Razaghi, and M. Mardi, "A mobile application for managing diabetic patients' nutrition: A food recommender system," *Archives of Iranian Medicine*, vol. 21, no. 10, pp. 466–472, 2018.
- [32] M. A. Khan, N. Sarwar, and N. F. Warraich, "Health-aware food recommendation system using ensemble learning for chronic disease patients," *Journal of Healthcare Engineering*, vol. 2022, pp. 1–12, 2022.
- [33] A. K. Gopalakrishnan, "A food recommendation system based on BMI, BMR, k-nn algorithm and a BPNN," *International Journal of Computer Applications*, vol. 175, no. 30, pp. 12–18, 2020.

- [34] I. Papastratis, D. Konstantinidis, P. Daras, and K. Dimitropoulos, “AI nutrition recommendation using a deep generative model and ChatGPT,” *Scientific Reports*, vol. 14, p. 14623, 2024.
- [35] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [36] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” *Advances in Neural Information Processing Systems*, vol. 30, pp. 3149–3157, 2017.
- [37] C. Iwendi, A. K. Bashir, A. Peshkar, R. Srivastava, J. M. Chatterjee, S. Pasupuleti, R. Mishra, S. Pillai, and O. Jo, “Diet recommendation system for chronic kidney disease patients using machine learning,” *Sensors*, vol. 21, no. 2, p. 394, 2021.
- [38] F. Pecune, S. Murali, V. Tsai, Y. Matsuyama, and J. Cassell, “A recommender system for healthy and personalized nutrition guidance,” *Proceedings of the Workshop on Health Recommender Systems*, pp. 14–20, 2020.
- [39] C. Trattner and D. Elswiler, “Investigating the healthiness of internet-sourced recipes: Implications for meal planning and recommender systems,” *Proceedings of the World Wide Web Conference*, pp. 489–498, 2020.
- [40] M. Ge, F. Ricci, and D. Massimo, “Health-aware food recommender system,” *Proceedings of the 9th ACM Conference on Recommender Systems*, pp. 333–334, 2015.
- [41] M. M. Hassan, M. G. R. Alam, M. Z. Uddin, S. Huda, A. Almogren, and G. Fortino, “Diet recommendation system for children with autism using machine learning,” *Future Generation Computer Systems*, vol. 111, pp. 889–898, 2020.
- [42] A. C. Paludo, D. C. L. Masquio, R. V. Patin, and R. C. Alves, “Ensemble methods for dietary assessment in chronic disease management,” *Nutrients*, vol. 13, no. 8, p. 2812, 2021.
- [43] Y. Cohen, R. Valdes-Mas, and E. Elinav, “The role of artificial intelligence in deciphering diet–disease relationships,” *Annual Review of Nutrition*, vol. 43, pp. 225–250, 2023.
- [44] T. De Pessemier, S. Dooms, and L. Martens, “Food recommender systems: Important factors and experiments,” *User Modeling and User-Adapted Interaction*, vol. 30, pp. 321–352, 2020.

- [45] P. Khanna, N. Agarwal, and A. Basu, “Precision nutrition for metabolic syndrome: An ensemble machine learning approach,” *Computers in Biology and Medicine*, vol. 149, p. 105997, 2022.